

SOLIDWORKS API (NET VBA C#) — Examples (14/50):

Create_Planar_Surface_Feature_Example_VB- Create_Ref_Plane_SSMember_Example_VBNET

SOLIDWORKS API Documentation (converted from CHM)

Create Planar Surface Feature Example (VB.NET)

This example shows how to create a planar surface feature.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with a boss extrusion.
' 2. Creates Surface-Radiate1 and Surface-Plane1 in the FeatureManager
'    design tree.
' 3. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim Part As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swSelData As SelectData
    Dim swRadiate As SurfaceRadiateFeatureData
    Dim swPlane As SurfacePlanarFeatureData
    Dim swFeat As Feature
    Dim swEnt As Entity
    Dim vRadEnt As Object
    Dim vBoundEnt As Object
    Dim swRadDirEnt As Entity
    Dim i As Integer
    Dim boolStatus As Boolean

```

Sub main()

```

    Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Part = swApp.ActiveDoc

    boolStatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.0448901407839529, 0.0279954694016864, 0.00466820674117181
    Part.SketchManager.InsertSketch(True)
    Part.ClearSelection2(True)
    boolStatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreference
    boolStatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPrefere
    Dim vSkLines As Object
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0555749908365768, 0.0329075527136081, 0, 0.0478203409524033, -0.03171
    Part.ClearSelection2(True)
    Part.SketchManager.InsertSketch(True)
    Part.ShowNamedView2("**Trimetric", 8)
    Part.SketchManager.InsertSketch(True)
    Part.ClearSelection2(True)
    boolStatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

```

Dim myFeature As Object

```

myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.

```

```

    boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0447337592343047, 0.0328467250718631, 0.00258132540182032, False, 2,
    boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0556265649287866, 0.0156695101210289, 0.0025672149453726, True, 2, No
    boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0140113588298618, -0.0317157034173761, 0.00254079743683633, True, 2,
    boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047780958393389, -0.00542256709667299, 0.00256078163948814, True, 2, 1
    boolStatus = Part.Extension.SelectByID2("", "FACE", 0.0478203409524554, -0.00305747564971171, 0.000546558985774936, True, 1,

```

```

    Part.InsertRadiateSurface(0.0254, False, False)

```

```

    swSelMgr = Part.SelectionManager
    swSelData = swSelMgr.CreateSelectData
    boolStatus = Part.Extension.SelectByID2("Surface-Radiate1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)

```

```

    swFeat = swSelMgr.GetSelectedObject6(1, -1)
    swRadiate = swFeat.GetDefinition

```

```

' Get radiate surface data
Debug.Print("File = " & Part.GetPathName)
Debug.Print(" " & swFeat.Name)
Debug.Print(" Distance: " & swRadiate.Distance * 1000.0# & " mm")
Debug.Print(" Flip? " & swRadiate.Flip)
Debug.Print(" Propagate to tangent faces? " & swRadiate.PropagateToTangentFaces)

```

```

' Roll back to get direction reference and radiated edges
boolStatus = swRadiate.AccessSelections(Part, Nothing)
swRadDirEnt = swRadiate.DirectionReference
Part.ClearSelection2(True)

```

```

vRadEnt = swRadiate.RadiatedEntities

```

```

Debug.Print("Type of radiated entities as defined in swSelectType_e:")
For i = 0 To swRadiate.GetRadiatedEntitiesCount - 1

    swEnt = vRadEnt(i)
    Debug.Print(" Radiated entity(" & i & ") = " & swEnt.GetType)

Next i

swRadiate.ReleaseSelectionAccess()

Part.ClearSelection2(True)
boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0150796523155918, 0.0330635737250304, 0.0277193529692568, False, 1, 1, 0)
boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0556924155575302, -0.00146678127401856, 0.0280574247206573, True, 1, 1, 0)
boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.00737846281163002, -0.0317262656324715, 0.0279565971788429, True, 1, 1, 0)
boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047950275020753, 0.00971418585908168, 0.0278100659315328, True, 1, Not

boolStatus = Part.InsertPlanarRefSurface()
Part.ClearSelection2(True)

boolStatus = Part.Extension.SelectByID2("Surface-Plane1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)

' Roll back to get the planar surface bounding edges
swFeat = swSelMgr.GetSelectedObject6(1, -1)
swPlane = swFeat.GetDefinition

boolStatus = swPlane.AccessSelections(Part, Nothing)
vBoundEnt = swPlane.BoundingEntities

Debug.Print("Types of planar surface bounding entities as defined in swSelectType_e:")
For i = 0 To swPlane.GetBoundingEntitiesCount - 1

    swEnt = vBoundEnt(i)
    Debug.Print(" Bounding entity(" & i & ") = " & swEnt.GetType)

Next i

swPlane.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Plane thru 3 Points In-context Example (VBA)

This example shows how to create a plane through 3 points in-context.

```

'-----
' Preconditions:
' 1. Open a fully resolved assembly.
' 2. Select a component.
' 3. Expand the selected component in the FeatureManager
'    design tree.
'
' Postconditions:
' 1. Creates a plane, passing through three points,
'    in the selected component.
' 2. Examine the selected component in the FeatureManager
'    design tree to verify that a 3D sketch and plane
'    were created.
'-----
Option Explicit

' Possible status values for AssemblyDoc::EditPart2
Public Enum swEditPartCommandStatus_e
    swEditPartFailure = -1
    swEditPartAsmMustBeSaved = -2
    swEditPartCompMustBeSelected = -3
    swEditPartCompMustBeResolved = -4
    swEditPartCompMustHaveWriteAccess = -5
    swEditPartSuccessful = 0
    swEditPartCompNotPositioned = &H1
End Enum

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swEditModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim swPart As SldWorks.PartDoc
    Dim swSketchPt1 As SldWorks.SketchPoint
    Dim swSketchPt2 As SldWorks.SketchPoint
    Dim swSketchPt3 As SldWorks.SketchPoint
    Dim swPlane As SldWorks.RefPlane
    Dim nRetVal As Long
    Dim nInfo As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swAssy = swModel

    ' Start in-context edit
    nRetVal = swAssy.EditPart2(True, False, nInfo)
    Debug.Assert swEditPartSuccessful = nRetVal

    Set swEditModel = swModel

    ' Turn off snapping
    swEditModel.SetAddToDB True

    ' Insert part/component 3D sketch in-context
    swEditModel.Insert3DSketch2 True

    ' Create points in part
    Set swSketchPt1 = swEditModel.CreatePoint2(0#, 0.02123307340457, 0.005485856156458)
    Set swSketchPt2 = swEditModel.CreatePoint2(0.04415646169588, 0.01166034702997, -0.00770979679615)
    Set swSketchPt3 = swEditModel.CreatePoint2(0#, -0.006247647329005, 0.007641244473859)

    ' Exit sketch but in assembly
    ' This gets you to editing part/component in-context
    swModel.Insert3DSketch2 True

    ' Restore snapping
    swEditModel.SetAddToDB False

    swModel.ClearSelection2 True

    bRet = swSketchPt1.Select4(True, swSelData): Debug.Assert bRet
    bRet = swSketchPt2.Select4(True, swSelData): Debug.Assert bRet
    bRet = swSketchPt3.Select4(True, swSelData): Debug.Assert bRet

    ' Create plane in part/component
    Set swPlane = swModel.CreatePlaneThru3Points3(True)
    Debug.Assert Not swPlane Is Nothing

    ' Go back to assembly
    ' End in-context edit
    swAssy.EditAssembly

End Sub

```

Create Plane using Curve and Point Example (VBA)

This example shows how to create a plane using a curve (edge) and a point.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly.  
' 2. Select an edge and a point.  
'  
' Postconditions:  
' 1. Creates a plane is created using the selected  
'    edge and point.  
' 2. Examine the graphics area and FeatureManager design  
'    tree.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swRefPlane As SldWorks.RefPlane  
    Dim swFeat As SldWorks.Feature  
    Dim vPlaneParam As Variant  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swRefPlane = swModel.CreatePlanePerCurveAndPassPoint3(True, True)  
    Set swFeat = swRefPlane  
    vPlaneParam = swRefPlane.GetRefPlaneParams  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "   Plane name = " & swFeat.Name  
    Debug.Print "      Origin = (" & vPlaneParam(0) * 1000# & ", " & vPlaneParam(1) * 1000# & ", " & vPlaneParam(2) * 1000# & ") mm"  
    Debug.Print "      Xvector = (" & vPlaneParam(3) & ", " & vPlaneParam(4) & ", " & vPlaneParam(5) & ")"  
    Debug.Print "      Normal  = (" & vPlaneParam(6) & ", " & vPlaneParam(7) & ", " & vPlaneParam(8) & ")"  
  
    swModel.EditRebuild3  
  
End Sub
```

Create Plots and Get Values Example (C#)

This example shows how to get the translational displacement and velocity values of a selected face and how to plot these values.

```
//-----
// Preconditions:
// 1. Start SOLIDWORKS Premium, which includes SOLIDWORKS Motion.
// 2. In SOLIDWORKS Design:
//   a. Start the SOLIDWORKS Motion Study add-in (in
//      SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Motion).
//   b. Open public_documents\samples\tutorial\motionstudies\valve_cam.sldasm.
//   c. Click the 1200 Motion Study tab in the lower-left corner of
//      the MotionManager.
//   d. Select Motion Analysis in the Type of Study list at the
//      upper-left corner of the MotionManager.
// 3. In the IDE:
//   a. Add a reference to the SOLIDWORKS Motion Study primary interop assembly
//      (right-click the name of the project in the Project Explorer >
//      click Add Reference > browse to install_dir\api\redist >
//      click SolidWorks.Interop.swmotionstudy.dll).
//   b. Open the Immediate window.
//
// Postconditions:
// 1. Selects two faces.
// 2. Calculates the motion analysis for the two selected faces.
// 3. Gets the translational displacement and translational velocity
//    values for the first selected face.
// 4. Creates plots of the translational displacement and translational
//    velocity.
// 5. Prints the names of the plot features to the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swmotionstudy;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace CreatePlotFeaturesCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MotionStudyManager MotionMgr = default(MotionStudyManager);
            MotionStudy MotionStudy = default(MotionStudy);
            CosmosMotionStudyResults MotionStudyResults = default(CosmosMotionStudyResults);
            bool status = false;
            MotionPlotFeatureData swSimPlotFeatureData = default(MotionPlotFeatureData);
            MotionPlotAxisFeatureData swSimPlotXAxisFeatureData = default(MotionPlotAxisFeatureData);
            MotionPlotAxisFeatureData[] swSimPlotYAxisFeatureData = new MotionPlotAxisFeatureData[2];
            Face2 swFace = default(Face2);
            Entity swEntity = default(Entity);
            object[] swFaceArray = new object[1];
            Entity[] swEntityArray = new Entity[1];
            MotionPlotFeatureOutput PlotOutput = default(MotionPlotFeatureOutput);
            Feature swPlotFeature1 = default(Feature);
            Feature swPlotFeature2 = default(Feature);
            double[] swXData = null;
            double[] swYData = null;
            string[] nameYAxis = new string[2];
            int i = 0;
            int j = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select the faces for which to calculate
            // motion analysis, then calculate it and
            // get the results
            swModel.ShowNamedView2("Right", 4);
            status = swModelDocExt.SelectByID2("", "FACE", 0.03426699306681, 0.03342024416822, 0.02599934303839, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 0.03047373686337, 0.006937653650944, 0.02566622869226, true, 0, null, 0);
            MotionMgr = (MotionStudyManager)swModelDocExt.GetMotionStudyManager();
            MotionStudy = (MotionStudy)MotionMgr.GetMotionStudy("1200");
            status = MotionStudy.Calculate();
            MotionStudyResults = (CosmosMotionStudyResults)MotionStudy.GetResults(4);

            // Create a plot feature data and create the x and y axes feature data
            swSimPlotFeatureData = (MotionPlotFeatureData)MotionStudyResults.CreatePlotFeatureData();
```

```

swSimPlotXAxisFeatureData = (MotionPlotAxisFeatureData)MotionStudyResults.CreatePlotXAxisFeatureData();
swSimPlotYAxisFeatureData[0] = (MotionPlotAxisFeatureData)MotionStudyResults.CreatePlotYAxisFeatureData();
swSimPlotYAxisFeatureData[1] = (MotionPlotAxisFeatureData)MotionStudyResults.CreatePlotYAxisFeatureData();

// Set the type of plots
nameYAxis[0] = "swMotionPlotAxisType_TRANS_DISP";
swSimPlotYAxisFeatureData[0].Type = (int)swMotionPlotAxisType_e.swMotionPlotAxisType_TRANS_DISP;
swSimPlotYAxisFeatureData[0].Component = 1;
nameYAxis[1] = "swMotionPlotAxisType_TRANS_VELOCITY";
swSimPlotYAxisFeatureData[1].Type = (int)swMotionPlotAxisType_e.swMotionPlotAxisType_TRANS_VELOCITY;
swSimPlotYAxisFeatureData[1].Component = 1;

// Get the entity whose motion you want analyzed
swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
swEntity = (Entity)swFace;
swEntityArray[0] = swEntity;

swSimPlotYAxisFeatureData[0].Entities = swEntityArray;
swSimPlotYAxisFeatureData[1].Entities = swEntityArray;

// Get the plot's x-axis and y-axes values
DispatchWrapper[] swYAxisArray = new DispatchWrapper[2];
swYAxisArray[0] = new DispatchWrapper(swSimPlotYAxisFeatureData[0]);
swYAxisArray[1] = new DispatchWrapper(swSimPlotYAxisFeatureData[1]);
PlotOutput = (MotionPlotFeatureOutput)MotionStudyResults.GetValues(swSimPlotFeatureData, swSimPlotXAxisFeatureData,
(swYAxisArray));
swXData = (double[])PlotOutput.GetAxis();

// Print the x-axis values and the y-axis translational
// displacement values and the y-axes translational velocity
// values to the Immediate window
Debug.Print("");
string x = "";
string y = "";
for (i = 0; i <= swYAxisArray.GetUpperBound(0); i++)
{
    Debug.Print("----- YAxis Type : " + nameYAxis[i]);
    swYData = (double[])PlotOutput.GetAxis(swSimPlotYAxisFeatureData[i]);
    for (j = 0; j <= swXData.GetUpperBound(0); j++)
    {
        x = swXData[j].ToString();
        y = swYData[j].ToString();
        Debug.Print(" (x, y) : (" + x + ", " + y + ")");
    }
}

// Insert and display the translational displacement plot
swPlotFeature1 = (Feature)MotionStudyResults.InsertPlotFeature(swSimPlotFeatureData, swSimPlotXAxisFeatureData,
swSimPlotYAxisFeatureData[0]);
Debug.Print("Name of plot feature: " + swPlotFeature1.Name);

// Insert and display the translational velocity plot
swPlotFeature2 = (Feature)MotionStudyResults.InsertPlotFeature(swSimPlotFeatureData, swSimPlotXAxisFeatureData,
swSimPlotYAxisFeatureData[1]);
Debug.Print("Name of plot feature: " + swPlotFeature2.Name);

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Create Plots and Get Values Example (VBA)

This example shows how to get the translational displacement and velocity values for a selected face and how to plot these values.

```

'-----
' Preconditions:
' 1. Start SOLIDWORKS Premium, which includes SOLIDWORKS Motion.
' 2. In SOLIDWORKS Design:
'   a. Start the SOLIDWORKS Motion Study add-in (click Tools >
'      Add-Ins > SOLIDWORKS Motion).
'   b. Open public_documents\samples\tutorial\motionstudies\valve_cam.sldasm.
'   c. Click the 1200 Motion Study tab in the lower-left corner of
'      the MotionManager.
'   d. Select Motion Analysis in the Type of Study list at the
'      upper-left corner of the MotionManager.
' 3. In the IDE:
'   a. Add a reference to the SOLIDWORKS MotionStudy type library.
'   b. Open the Immediate window.
'
' Postconditions:
' 1. Selects two faces.
' 2. Calculates the motion analysis for the two selected faces.
' 3. Gets the translational displacement and translational velocity
'    values for the first selected face.
' 4. Creates plots of the translational displacement and translational
'    velocity.
' 5. Prints the names of the plot features to the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim MotionMgr As SwMotionStudy.MotionStudyManager
    Dim MotionStudy As SwMotionStudy.MotionStudy
    Dim MotionStudyResults As SwMotionStudy.CosmosMotionStudyResults
    Dim status As Boolean
    Dim swSimPlotFeatureData As SldWorks.MotionPlotFeatureData
    Dim swSimPlotXAxisFeatureData As SldWorks.MotionPlotAxisFeatureData
    Dim swSimPlotYAxisFeatureData(0 To 1) As SldWorks.MotionPlotAxisFeatureData
    Dim swFace(0 To 0) As SldWorks.Face2
    Dim swFaceArray As Variant
    Dim swYAxisArray As Variant
    Dim PlotOutput As SwMotionStudy.MotionPlotFeatureOutput
    Dim swPlotFeature1 As SldWorks.Feature
    Dim swPlotFeature2 As SldWorks.Feature
    Dim swXData As Variant
    Dim swYData As Variant
    Dim nameYAxis(0 To 1) As String
    Dim i As Long
    Dim j As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    ' Select the faces for which to calculate
    ' motion analysis, then calculate it and
    ' get the results
    swModel.ShowNamedView2 "Right", 4
    status = swModelDocExt.SelectByID("", "FACE", 0.03426699306681, 0.03342024416822, 0.02599934303839, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", 0.03047373686337, 0.006937653650944, 0.02566622869226, True, 0, Nothing, 0)
    Set MotionMgr = swModelDocExt.GetMotionStudyManager()
    Set MotionStudy = MotionMgr.GetMotionStudy("1200")
    status = MotionStudy.Calculate
    Set MotionStudyResults = MotionStudy.GetResults(4)

    ' Create a plot feature data and create the x and y axes feature data
    Set swSimPlotFeatureData = MotionStudyResults.CreatePlotFeatureData()
    Set swSimPlotXAxisFeatureData = MotionStudyResults.CreatePlotXAxisFeatureData()
    Set swSimPlotYAxisFeatureData(0) = MotionStudyResults.CreatePlotYAxisFeatureData()
    Set swSimPlotYAxisFeatureData(1) = MotionStudyResults.CreatePlotYAxisFeatureData()

    ' Set the type of plots
    nameYAxis(0) = "swMotionPlotAxisType_TRANS_DISP"
    swSimPlotYAxisFeatureData(0).Type = SwConst.swMotionPlotAxisType_TRANS_DISP
    swSimPlotYAxisFeatureData(0).Component = 1
    nameYAxis(1) = "swMotionPlotAxisType_TRANS_VELOCITY"
    swSimPlotYAxisFeatureData(1).Type = SwConst.swMotionPlotAxisType_TRANS_VELOCITY
    swSimPlotYAxisFeatureData(1).Component = 1

    ' Get the entity whose motion you want analyzed
    Set swFace(0) = swSelMgr.GetSelectedObject6(1, -1)
    swFaceArray = swFace
    swSimPlotYAxisFeatureData(0).Entities = swFaceArray
    swSimPlotYAxisFeatureData(1).Entities = swFaceArray

```

```

' Get the plot's x-axis and y-axes values
swYAxisArray = swSimPlotYAxisFeatureData
Set PlotOutput = MotionStudyResults.GetValues(swSimPlotFeatureData, swSimPlotXAxisFeatureData, (swYAxisArray))
swXData = PlotOutput.GetXAxis()

' Print the x-axis values and the y-axis translational
' displacement values and the y-axes translational velocity
' values to the Immediate window
Debug.Print ""
For i = 0 To UBound(swYAxisArray)
    Debug.Print "----- YAxis Type : " & nameYAxis(i)
    swYData = PlotOutput.GetYAxis(swSimPlotYAxisFeatureData(i))
    For j = 0 To UBound(swXData)
        Debug.Print " (x, y) : (" & Strings.Format(swXData(j)) & ", " & swYData(j) & ")"
    Next j
Next i

' Insert and display the translational displacement plot
Set swPlotFeature1 = MotionStudyResults.InsertPlotFeature(swSimPlotFeatureData, swSimPlotXAxisFeatureData, swSimPlotYAxisFeatureData(0))
Debug.Print "Name of plot feature: " & swPlotFeature1.Name

' Insert and display the translational velocity plot
Set swPlotFeature2 = MotionStudyResults.InsertPlotFeature(swSimPlotFeatureData, swSimPlotXAxisFeatureData, swSimPlotYAxisFeatureData(1))
Debug.Print "Name of plot feature: " & swPlotFeature2.Name

End Sub

```

Create Plots and Get Values Example (VB.NET)

This example shows how to get the translational displacement and velocity values for a selected face and how to plot these values.

```

'-----
' Preconditions:
' 1. Start SOLIDWORKS Premium, which includes SOLIDWORKS Motion.
' 2. In SOLIDWORKS Design:
'   a. Start the SOLIDWORKS Motion Study add-in (click Tools >
'      Add-Ins > SOLIDWORKS Motion).
'   b. Open public_documents\samples\tutorial\motionstudies\valve_cam.sldasm.
'   c. Click the 1200 Motion Study tab in the lower-left corner of
'      the MotionManager.
'   d. Select Motion Analysis in the Type of Study list at the
'      upper-left corner of the MotionManager.
' 3. In the IDE:
'   a. Add a reference to the SOLIDWORKS Motion Study primary interop assembly
'      (right-click the name of the project in the Project Explorer >
'      click Add Reference > browse to install_dir\api\redist >
'      click SolidWorks.Interop.swmotionstudy.dll).
'   b. Open the Immediate window.
'
' Postconditions:
' 1. Selects two faces.
' 2. Calculates the motion analysis for the two selected faces.
' 3. Gets the translational displacement and translational velocity
'    values for the first selected face.
' 4. Creates plots of the translational displacement and translational
'    velocity.
' 5. Prints the names of the plot features to the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swmotionstudy
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim MotionMgr As MotionStudyManager
        Dim MotionStudy As MotionStudy
        Dim MotionStudyResults As CosmosMotionStudyResults
        Dim status As Boolean
        Dim swSimPlotFeatureData As MotionPlotFeatureData
        Dim swSimPlotXAxisFeatureData As MotionPlotAxisFeatureData
        Dim swSimPlotYAxisFeatureData(0 To 1) As MotionPlotAxisFeatureData
        Dim swFace(0 To 0) As Face2
        Dim swFaceArray() As Object
        Dim swYAxisArray() As Object
        Dim PlotOutput As MotionPlotFeatureOutput
        Dim swPlotFeature1 As Feature
        Dim swPlotFeature2 As Feature
        Dim swXData As Object
        Dim swYData As Object
        Dim nameYAxis(0 To 1) As String
        Dim i As Integer
        Dim j As Integer

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension

        ' Select the faces for which to calculate
        ' motion analysis, then calculate it and
        ' get the results
        swModel.ShowNamedView2("Right", 4)
        status = swModelDocExt.SelectByID2("", "FACE", 0.03426699306681, 0.03342024416822, 0.02599934303839, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.03047373686337, 0.006937653650944, 0.02566622869226, True, 0, Nothing, 0)
        MotionMgr = swModelDocExt.GetMotionStudyManager()
        MotionStudy = MotionMgr.GetMotionStudy("1200")
        status = MotionStudy.Calculate
        MotionStudyResults = MotionStudy.GetResults(4)

        ' Create a plot feature data and create the x and y axes feature data
        swSimPlotFeatureData = MotionStudyResults.CreatePlotFeatureData()
        swSimPlotXAxisFeatureData = MotionStudyResults.CreatePlotXAxisFeatureData()
        swSimPlotYAxisFeatureData(0) = MotionStudyResults.CreatePlotYAxisFeatureData()
        swSimPlotYAxisFeatureData(1) = MotionStudyResults.CreatePlotYAxisFeatureData()

        ' Set the type of plots
        nameYAxis(0) = "swMotionPlotAxisType_TRANS_DISP"
        swSimPlotYAxisFeatureData(0).Type = swMotionPlotAxisType_e.swMotionPlotAxisType_TRANS_DISP
    
```

```

swSimPlotYAxisFeatureData(0).Component = 1
nameYAxis(1) = "swMotionPlotAxisType_TRANS_VELOCITY"
swSimPlotYAxisFeatureData(1).Type = swMotionPlotAxisType_e.swMotionPlotAxisType_TRANS_VELOCITY
swSimPlotYAxisFeatureData(1).Component = 1

' Get the entity whose motion you want analyzed
swFace(0) = swSelMgr.GetSelectedObject6(1, -1)
swFaceArray = swFace
swSimPlotYAxisFeatureData(0).Entities = swFaceArray
swSimPlotYAxisFeatureData(1).Entities = swFaceArray

' Get the plot's x-axis and y-axes values
swYAxisArray = swSimPlotYAxisFeatureData
PlotOutput = MotionStudyResults.GetValues(swSimPlotFeatureData, swSimPlotXAxisFeatureData, (swYAxisArray))
swXData = PlotOutput.GetAxis()

' Print the x-axis values and the y-axis translational
' displacement values and the y-axes translational velocity
' values to the Immediate window
Debug.Print("")
For i = 0 To UBound(swYAxisArray)
    Debug.Print("----- YAxis Type : " & nameYAxis(i))
    swYData = PlotOutput.GetYAxis(swSimPlotYAxisFeatureData(i))
    For j = 0 To UBound(swXData)
        Debug.Print(" (x, y) : (" & Strings.Format(swXData(j) & ", " & swYData(j) & ")")
    Next j
Next i

' Insert and display the translational displacement plot
swPlotFeature1 = MotionStudyResults.InsertPlotFeature(swSimPlotFeatureData, swSimPlotXAxisFeatureData,
swSimPlotYAxisFeatureData(0))
Debug.Print("Name of plot feature: " & swPlotFeature1.Name)

' Insert and display the translational velocity plot
swPlotFeature2 = MotionStudyResults.InsertPlotFeature(swSimPlotFeatureData, swSimPlotXAxisFeatureData,
swSimPlotYAxisFeatureData(1))
Debug.Print("Name of plot feature: " & swPlotFeature2.Name)

End Sub

''' <summary>
''' The Sldworks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Point Length Structure System Member Example (C#)

This example shows how to create a structure system member that originates at a point and extends to a specified end condition.

```
// =====
// Preconditions:
// 1. Ensure the specified part template exists.
// 2. Open an Immediate window.
// 3. Press F5.
//
// Postconditions:
// 1. Creates Sketch1, Sketch2, and Sketch3 in a new part.
// 2. Configures the start/end extensions and the member profile.
// 3. Configures the length end condition for the structure members.
// 4. Selects three sketch points.
// 5. Specifies the length.
// 6. Creates a Structure System1 with three structure system point length members.
// 7. Inspect the structure system in the graphics area.
// 8. Press F5 to edit the structure system.
// 9. Gets one structure system member.
// 10. Changes the length and member extensions in directions 1 and 2.
// 11. Modifies the structure system.
// 12. Inspect the changes in the graphics area.
// =====
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
namespace PointLength_CSharp
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private ModelDocExtension modDocExt;
        private StructureSystemMemberFeatureData structMemDef;
        private StructureSystemMemberProfile profDef;
        private PrimaryStructuralMemberFeatureData PrimDef;
        private PrimaryMemberPointLengthFeatureData memPointLengthDef;
        private object[] startPoints = new object[3];
        private bool stat;
        private StructureSystemMemberFeatureData[] memData = new StructureSystemMemberFeatureData[1];
        private object[] memDataArray;
        private Feature systemFeature;
        private StructureSystemFolder structSysModDef;
        private object[] outProfiles;
        private StructureSystemMemberFeatureData MemberData;
        private object[] memberArray;
        private Feature profileGrpFeat;
        private ProfileGroupFolder profileGrp;
        private StructureSystemMemberFeatureData memToChange;
        private PrimaryMemberPointLengthFeatureData pointLengthMember;
        private Feature memberFeat;
        private int i;
        private int j;
```

```

public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize;
    ArraySize = Objects.GetUpperBound(0);
    var d = new Dispatcher[ArraySize + 1];
    int ArrayIndex;
    var loopTo = ArraySize;
    for (ArrayIndex = 0; ArrayIndex <= loopTo; ArrayIndex++)
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    return d;
}

public void Main()
{
    double swSheetWidth;
    swSheetWidth = 0d;
    double swSheetHeight;
    swSheetHeight = 0d;
    modDoc = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0,
swSheetWidth, swSheetHeight);
    PartDoc swPart;
    swPart = (PartDoc)modDoc;
    modDoc = (ModelDoc2)swApp.ActiveDoc;
    modDocExt = modDoc.Extension;
    swSelMgr = (SelectionMgr)modDoc.SelectionManager;
    modDoc.SketchManager.InsertSketch(true);
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", -0.0378286223842605d, 0.0426130572878663d, -0.00898903374939048d,
false, 0, null, 0);
    modDoc.ClearSelection2(true);
    SketchPoint skPoint;
    skPoint = modDoc.SketchManager.CreatePoint(-0.040799d, 0.008094d, 0d);
    modDoc.ClearSelection2(true);
    modDoc.SketchManager.InsertSketch(true);
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.SketchManager.InsertSketch(true);
    skPoint = modDoc.SketchManager.CreatePoint(-0.023183d, 0.004864d, 0d);
    modDoc.ClearSelection2(true);
    modDoc.SketchManager.InsertSketch(true);
    stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.SketchManager.InsertSketch(true);
    skPoint = modDoc.SketchManager.CreatePoint(-0.020877d, 0.010539d, 0d);
    modDoc.ClearSelection2(true);
    modDoc.SketchManager.InsertSketch(true);
    modDoc.ClearSelection2(true);
    modDoc.ShowNamedView2("Trimetric", 8);
    modDoc.ViewZoomtofit2();

    // Configure a structure system point length member
    structMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(2);
    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
structMemDef.StructureSystemMemberType);
    structMemDef.StartEndExtendD1 = 1.0d;
    Debug.Print("Extend length in direction 1: " + structMemDef.StartEndExtendD1);
    structMemDef.StartEndExtendD2 = 2.0d;
    Debug.Print("Extend length in direction 2: " + structMemDef.StartEndExtendD2);
    profDef = (StructureSystemMemberProfile)structMemDef.MemberProfile;
    profDef.ProfileStandard = "ansi inch";
    profDef.ProfileType = "c channel";
    profDef.ProfileSize = "3 x 5";
    PrimDef = (PrimaryStructuralMemberFeatureData)structMemDef;
    Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " +
PrimDef.PrimaryMemberType);
    memPointLengthDef = (PrimaryMemberPointLengthFeatureData)PrimDef;
    memPointLengthDef.EndCondition = 0;

```

```

// Set three points that define three structure system point length members
stat = modDocExt.SelectByID2("Point1@Sketch3", "EXTSKETCHPOINT", -0.020877d, 0.010539d, 0, false, 0, null, 0);
stat = modDocExt.SelectByID2("Point1@Sketch1", "EXTSKETCHPOINT", -0.040799d, 0.008094d, 0, true, 0, null, 0);
stat = modDocExt.SelectByID2("Point1@Sketch2", "EXTSKETCHPOINT", -0.023183d, 0.004864d, 0, true, 0, null, 0);
startPoints[0] = swSelMgr.GetSelectedObject6(1, 0);
startPoints[1] = swSelMgr.GetSelectedObject6(2, 0);
startPoints[2] = swSelMgr.GetSelectedObject6(3, 0);
DispatchWrapper[] dArray;
dArray = ObjectArrayToDispatchWrapperArray(startPoints);
stat = memPointLengthDef.SetPoints(dArray);
Debug.Print("Start points set successfully: " + stat);

// Set the length for each member
memPointLengthDef.Length = 5;
memPointLengthDef.ReverseDirection = false;
memData[0] = structMemDef;
memDataArray = memData;
systemFeature = modDocExt.CreateStructureSystem(memDataArray, null);
modDoc.ViewZoomtofit2();
Debugger.Break();
structSysModDef = (StructureSystemFolder)systemFeature.GetSpecificFeature2();
Debug.Print("Number of structure system profile groups: " + structSysModDef.GetProfileGroupFoldersCount());
outProfiles = (object[])structSysModDef.GetProfileGroupFolders();
if (outProfiles.Length > 0)
{
    var loopTo = outProfiles.GetUpperBound(0);
    for (I = 0; I <= loopTo; I++)
    {
        profileGrpFeat = (Feature)outProfiles[I];
        profileGrp = (ProfileGroupFolder)profileGrpFeat.GetSpecificFeature2();
        if (profileGrp is object)
        {
            Debug.Print("Number of members inside profile group " + profileGrp.GetStructureSystemMemberCount());
            memberArray = (object[])profileGrp.GetStructureSystemMembers();
            if (memberArray.Length > 0)
            {
                var loopTo1 = memberArray.GetUpperBound(0);
                for (j = 0; j <= loopTo1; j++)
                {
                    MemberData = (StructureSystemMemberFeatureData)memberArray[j]; // Get one structure system member
                    break;
                }
            }
        }
    }
}

// Changing one member only
memToChange = MemberData;
Debug.Print("Change member length and extensions in directions 1 and 2 for one member only");
memToChange.StartEndExtendD1 = 0.1d;
memToChange.StartEndExtendD2 = 0.1d;
pointLengthMember = (PrimaryMemberPointLengthFeatureData)MemberData;
pointLengthMember.Length = 1;

// Save edits
memberFeat = memToChange.GetFeature();
stat = memberFeat.ModifyDefinition(MemberData, modDoc, null);
}

// The SldWorks swApp variable is pre-assigned for you.

```

to change

```
public SldWorks swApp;
```

```
}
```

```
}
```

Create Point Length Structure System Member Example (VB.NET)

This example shows how to create a structure system member that originates at a point and extends to a specified end condition.

```
'=====
'Preconditions:
'1. Ensure the specified part template exists.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
' 1. Creates Sketch1, Sketch2, and Sketch3 in a new part.
' 2. Configures the start/end extensions and the member profile.
' 3. Configures the length end condition for the structure members.
' 4. Selects three sketch points.
' 5. Specifies the length.
' 6. Creates a Structure System1 with three structure system point length members.
' 7. Inspect the structure system in the graphics area.
' 8. Press F5 to edit the structure system.
' 9. Gets one structure system member.
'10. Changes the length and member extensions in directions 1 and 2.
'11. Modifies the structure system.
'12. Inspect the changes in the graphics area.
'=====

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
    Dim modDoc As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim modDocExt As ModelDocExtension
    Dim structMemDef As StructureSystemMemberFeatureData
    Dim profDef As StructureSystemMemberProfile
    Dim PrimDef As PrimaryStructuralMemberFeatureData
    Dim memPointLengthDef As PrimaryMemberPointLengthFeatureData
    Dim startPoints(2) As Object
    Dim stat As Boolean
    Dim memData(0) As StructureSystemMemberFeatureData
    Dim memDataArray() As Object
    Dim systemFeature As Feature
    Dim structSysModDef As StructureSystemFolder
    Dim outProfiles() As Object
    Dim MemberData As StructureSystemMemberFeatureData
    Dim memberArray() As Object
    Dim profileGrpFeat As Feature
    Dim profileGrp As ProfileGroupFolder
    Dim memTochange As StructureSystemMemberFeatureData
    Dim pointLengthMember As PrimaryMemberPointLengthFeatureData
    Dim memberFeat As Feature
    Dim i As Integer
    Dim j As Integer

    Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
```

```

Dim ArraySize As Integer
ArraySize = Objects.GetUpperBound(0)
Dim d(ArraySize) As Dispatcher
Dim ArrayIndex As Integer
For ArrayIndex = 0 To ArraySize
    d(ArrayIndex) = New Dispatcher(Objects(ArrayIndex))
Next
Return d
End Function

Sub main()

    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    modDoc = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0, swSheetWidth,
swSheetHeight)
    Dim swPart As PartDoc
    swPart = modDoc
    modDoc = swApp.ActiveDoc

    modDocExt = modDoc.Extension
    swSelMgr = modDoc.SelectionManager

    modDoc.SketchManager.InsertSketch(True)
    stat = modDocExt.SelectByID("Front Plane", "PLANE", -0.0378286223842605, 0.0426130572878663, -0.00898903374939048, False,
0, Nothing, 0)
    modDoc.ClearSelection2(True)
    Dim skPoint As SketchPoint
    skPoint = modDoc.SketchManager.CreatePoint(-0.040799, 0.008094, 0#)
    modDoc.ClearSelection2(True)
    modDoc.SketchManager.InsertSketch(True)
    stat = modDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.SketchManager.InsertSketch(True)
    skPoint = modDoc.SketchManager.CreatePoint(-0.023183, 0.004864, 0#)
    modDoc.ClearSelection2(True)
    modDoc.SketchManager.InsertSketch(True)
    stat = modDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.SketchManager.InsertSketch(True)
    skPoint = modDoc.SketchManager.CreatePoint(-0.020877, 0.010539, 0#)
    modDoc.ClearSelection2(True)
    modDoc.SketchManager.InsertSketch(True)

    modDoc.ClearSelection2(True)
    modDoc.ShowNamedView2("**Trimetric", 8)
    modDoc.ViewZoomtofit2

    'Configure a structure system point length member
    structMemDef = modDocExt.CreateStructureSystemMemberData(2)

    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

    structMemDef.StartEndExtendD1 = 1.0#
    Debug.Print("Extend length in direction 1: " & structMemDef.StartEndExtendD1)
    structMemDef.StartEndExtendD2 = 2.0#
    Debug.Print("Extend length in direction 2: " & structMemDef.StartEndExtendD2)

    profDef = structMemDef.MemberProfile

    profDef.ProfileStandard = "ansi inch"
    profDef.ProfileType = "c channel"
    profDef.ProfileSize = "3 x 5"

```

```

PrimDef = structMemDef
Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
PrimDef.PrimaryMemberType)

memPointLengthDef = PrimDef

memPointLengthDef.EndCondition = 0

'Set three points that define three structure system point length members
stat = modDocExt.SelectByID2("Point1@Sketch3", "EXTSKETCHPOINT", -0.020877, 0.010539, 0, False, 0, Nothing, 0)
stat = modDocExt.SelectByID2("Point1@Sketch1", "EXTSKETCHPOINT", -0.040799, 0.008094, 0, True, 0, Nothing, 0)
stat = modDocExt.SelectByID2("Point1@Sketch2", "EXTSKETCHPOINT", -0.023183, 0.004864, 0, True, 0, Nothing, 0)

startPoints(0) = swSelMgr.GetSelectedObject6(1, 0)
startPoints(1) = swSelMgr.GetSelectedObject6(2, 0)
startPoints(2) = swSelMgr.GetSelectedObject6(3, 0)

Dim dArray() As DispatchWrapper
dArray = ObjectArrayToDispatchWrapperArray(startPoints)

stat = memPointLengthDef.SetPoints(dArray)
Debug.Print("Start points set successfully: " & stat)

'Set the length for each member
memPointLengthDef.Length = 5
memPointLengthDef.ReverseDirection = False

memData(0) = structMemDef

memDataArray = memData

systemFeature = modDocExt.CreateStructureSystem(memDataArray, Nothing)

modDoc.ViewZoomtofit2

Stop

structSysModDef = systemFeature.GetSpecificFeature2
Debug.Print("Number of structure system profile groups: " & structSysModDef.GetProfileGroupFoldersCount)

outProfiles = structSysModDef.GetProfileGroupFolders()

If (outProfiles.Length > 0) Then
    For I = 0 To UBound(outProfiles)
        profileGrpFeat = outProfiles(I)
        profileGrp = profileGrpFeat.GetSpecificFeature2
        If Not profileGrp Is Nothing Then
            Debug.Print("Number of members inside profile group " & profileGrp.GetStructureSystemMemberCount)
            memberArray = profileGrp.GetStructureSystemMembers()
            If (memberArray.Length > 0) Then
                For j = 0 To UBound(memberArray)
                    MemberData = memberArray(j) 'Get one structure system member to change
                Exit For
            Next
        End If
    End If
Next
End If

'Changing one member only
memTochange = MemberData

```

```
Debug.Print("Change member length and extensions in directions 1 and 2 for one member only")
memToChange.StartEndExtendD1 = 0.1
memToChange.StartEndExtendD2 = 0.1

pointLengthMember = MemberData
pointLengthMember.Length = 1

'Save edits
memberFeat = memToChange.GetFeature()
stat = memberFeat.ModifyDefinition(MemberData, modDoc, Nothing)
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Polygon Example (C#)

This example shows how to create a polygon.

```
//-----  
// Preconditions: Open a part document.  
//  
// Postconditions:  
// 1. Inserts a sketch.  
// 2. Creates a six-sided polygon, circumscribed with a  
//    construction circle.  
// 3. Examine the graphics area.  
//-----  
using Microsoft.VisualBasic;  
  
using System;  
  
using System.Collections;  
  
using System.Collections.Generic;  
  
using System.Data;  
  
using System.Diagnostics;  
  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
  
using System.Runtime.InteropServices;  
  
namespace CreatePolygon_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
        ModelDocExtension swModelDocExt;  
        SketchManager swSketchMgr;  
        Object[]  
        polygon;  
  
        public void Main()  
        {  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            swModelDocExt = swModel.Extension;  
  
            swSketchMgr = swModel.SketchManager;  
  
            swSketchMgr.InsertSketch(false);  
  
            polygon = (Object[])swSketchMgr.CreatePolygon(-2.5,  
0.88, 0.0, -2.21, -2.13, 0.0, 6, false);  
  
            swModel.ViewZoomtofit2();  
  
            // Set the selection mode to  
            default  
  
            swModel.SetPickMode();  
  
            swSketchMgr.InsertSketch(true);  
  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Create Polygon Example (VBA)

This example shows how to create a polygon.

```
'-----  
' Preconditions: Open a part document.  
'  
' Postconditions:  
' 1. Inserts a sketch.  
' 2. Creates a six-sided polygon, circumscribed with a  
'    construction circle.  
' 3. Examine the graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchMgr As SldWorks.SketchManager  
Dim polygon() As SldWorks.SketchSegment  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSketchMgr = swModel.SketchManager  
  
    swSketchMgr.InsertSketch False  
    polygon = swSketchMgr.CreatePolygon(-2.5, 0.88, 0#, -2.21, -2.13, 0#, 6, False)  
  
    swModel.ViewZoomtofit2  
    ' Set the selection mode to default  
    swModel.SetPickMode  
    swSketchMgr.InsertSketch True  
End Sub
```

Create Polygon Example (VB.NET)

This example shows how to create a polygon.

```
'-----  
' Preconditions: Open a part document.  
'  
' Postconditions:  
' 1. Inserts a sketch.  
' 2. Creates a six-sided polygon, circumscribed with a  
'    construction circle.  
' 3. Examine the graphics area.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System.Runtime.InteropServices  
  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Dim swModel As ModelDoc2  
  
    Dim swModelDocExt As ModelDocExtension  
  
    Dim swSketchMgr As SketchManager  
  
    Dim polygon() As Object  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
  
        swModelDocExt = swModel.Extension  
  
        swSketchMgr = swModel.SketchManager  
  
        swSketchMgr.InsertSketch(False)  
  
        polygon = swSketchMgr.CreatePolygon(-2.5, 0.88, 0.0#, -2.21,  
-2.13, 0.0#, 6, False)  
  
        swModel.ViewZoomtofit2()  
  
        ' Set the selection mode to default  
        swModel.SetPickMode()  
  
        swSketchMgr.InsertSketch(True)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Create and Edit Profile Center Mate Example (C#)

This example shows how to create and edit a profile center mate.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo2.sldasm
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a profile center mate.
// 2. Press F5 to continue.
// 3. Changes the mate alignment of the mate.
// 4. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ProfileCenterMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private AssemblyDoc swAssy;
        private ProfileCenterMateFeatureData swProfileMateData;
        private MateFeatureData swMateData;
        private SelectionMgr swSelMgr;
        private bool boolstatus;
        private Feature feat;
        private Face2[] facesPC = new Face2[2];
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swMateData = (MateFeatureData)swAssy.CreateMateData(24); //Profile center mate

            if (swMateData.TypeName == (int)swMateType_e.swMatePROFILECENTER)
            {
                swProfileMateData = (ProfileCenterMateFeatureData)swMateData;

                swProfileMateData.MateAlignment = (int)swMateAlign_e.swMateAlignALIGNED;
                Debug.Print("Profile Center mate alignment is " + swProfileMateData.MateAlignment);

                swProfileMateData.FlipDimension = false;
                Debug.Print("ProfileCenter flip dimension is " + swProfileMateData.FlipDimension);

                swProfileMateData.LockRotation = false;
                Debug.Print("Profile center mate lock rotation is " + swProfileMateData.LockRotation);

                swProfileMateData.OffsetDistance = 0.0254;
                Debug.Print("Profile center mate offset distance is " + swProfileMateData.OffsetDistance);

                boolstatus = swModel.Extension.SelectByRay(0.0152510997612296, 0.0525002489357007, 0.132234612849345, -0.27184406061, -0.30846884126);
                boolstatus = swModel.Extension.SelectByRay(0.136053581313973, 0.0198237342244454, 0.0953905211266601, -0.30846884126);

                facesPC[0] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
                facesPC[1] = (Face2)swSelMgr.GetSelectedObject6(2, -1);

                object vFacesPC = facesPC;

                swProfileMateData.EntitiesToMate = vFacesPC;

                feat = (Feature)swAssy.CreateMate(swProfileMateData);
            }

            swModel.GraphicsRedraw2();

            System.Diagnostics.Debugger.Break();

            feat = swModel.Extension.GetLastFeatureAdded();

            Debug.Print("Feature GetTypeName2 of mate created is " + feat.GetTypeName2());
            swMateData = (MateFeatureData)feat.GetDefinition();

            if (swMateData.TypeName == (int)swMateType_e.swMatePROFILECENTER)
            {

```

```

        swProfileMateData = (ProfileCenterMateFeatureData)swMateData;
        Debug.Print("Profile center mate alignment is " + swProfileMateData.MateAlignment);
        if (swProfileMateData.MateAlignment == (int)swMateAlign_e.swMateAlignALIGNED)
        {
            swProfileMateData.MateAlignment = (int)swMateAlign_e.swMateAlignANTI_ALIGNED;
        }
        else
        {
            swProfileMateData.MateAlignment = (int)swMateAlign_e.swMateAlignALIGNED;
        }
        Debug.Print("Profile center mate alignment changed to " + swProfileMateData.MateAlignment);
        boolstatus = feat.ModifyDefinition(swProfileMateData, swAssy, null);
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create and Edit Profile Center Mate Example (VBA)

This example shows how to create and edit a profile center mate.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo2.sldasm
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a profile center mate.
' 2. Press F5 to continue.
' 3. Changes the mate alignment of the mate.
' 4. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swProfileMateData As SldWorks.ProfileCenterMateFeatureData
Dim swMateData As SldWorks.MateFeatureData
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim feat As SldWorks.Feature
Dim facesPC(1) As SldWorks.Face2

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager

    Set swMateData = swAssy.CreateMateData(24) 'Profile center mate

    If (swMateData.TypeName = SwConst.swMateType_e.swMatePROFILECENTER) Then

        Set swProfileMateData = swMateData

        swProfileMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED
        Debug.Print "Profile Center mate alignment is " & swProfileMateData.MateAlignment

        swProfileMateData.FlipDimension = False
        Debug.Print "ProfileCenter flip dimension is " & swProfileMateData.FlipDimension

        swProfileMateData.LockRotation = False
        Debug.Print "Profile center mate lock rotation is " & swProfileMateData.LockRotation

        swProfileMateData.OffsetDistance = 0.0254
        Debug.Print "Profile center mate offset distance is " & swProfileMateData.OffsetDistance

        boolstatus = swModel.Extension.SelectByRay(1.52510997612296E-02, 5.25002489357007E-02, 0.132234612849345,
        -0.271844060659921, -0.167859116984408, -0.947588583473408, 7.08485696755524E-04, 2, True, 1, 0)
        boolstatus = swModel.Extension.SelectByRay(0.136053581313973, 1.98237342244454E-02, 9.53905211266601E-02,
        -0.30846884126036, 0.319767565972896, -0.895877043864425, 7.08485696755524E-04, 2, False, 0, 0)

        Set facesPC(0) = swSelMgr.GetSelectedObject6(1, -1)
        Set facesPC(1) = swSelMgr.GetSelectedObject6(2, -1)

        Dim vFacesPC As Variant
        vFacesPC = facesPC

        swProfileMateData.EntitiesToMate = vFacesPC

        Set feat = swAssy.CreateMate(swProfileMateData)

    End If

    swModel.GraphicsRedraw2

    Stop

    Set feat = swModel.Extension.GetLastFeatureAdded

    Debug.Print "Feature GetTypeName2 of mate created is " & feat.GetTypeName2
    Set swMateData = feat.GetDefinition

```

```
If (swMateData.TypeName = SwConst.swMateType_e.swMatePROFILECENTER) Then
    Set swProfileMateData = swMateData

    Debug.Print "Profile center mate alignment is " & swProfileMateData.MateAlignment

    If (swProfileMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED) Then
        swProfileMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignANTI_ALIGNED
    Else
        swProfileMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED
    End If

    Debug.Print "Profile center mate alignment changed to " & swProfileMateData.MateAlignment

    boolstatus = feat.ModifyDefinition(swProfileMateData, swAssy, Nothing)

End If
End Sub
```

Create Projection Split Line Feature Example (C#)

This example shows how to create a projection split line feature.

```
//-----
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new model document with a feature extrusion, reference plane,
//    and sketch of an ellipse.
// 2. Creates Split Line1 in the FeatureManager design tree.
// 3. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertSplitLineFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SketchSegment skSegment;
        RefPlane myRefPlane;
        SelectionMgr swSelMgr;
        SplitLineFeatureData swSplitLine;
        object vSkLines;
        Feature myFeature;
        bool boolstatus;
        int longstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2015\\templates\\Part.prt", 0, 0, 0);
            Part = (ModelDoc2)swApp.ActiveDoc;

            skSegment = Part.SketchManager.CreateEllipse(-0.0212512457655407, 0.0122505076014363, 0, 0.00277468345541365, 0.00705202);
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);

            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);

            myRefPlane = (RefPlane)Part.FeatureManager.InsertRefPlane(8, 0.01778, 0, 0, 0, 0);
            Part.ClearSelection2(true);

            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", -0.0407148636658249, 0.0247341229458697, 0.0194913387248102,
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swl
            boolstatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int

            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0625406077424486, 0.0297244912047745, 0, 0.0584903577919818, -0.6
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 4, null, 0);

            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);
            ((SelectionMgr)(Part.SelectionManager)).EnableContourSelection = false;

            boolstatus = Part.Extension.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", -0.0143044793836914, 0.00334438727079956, 0, true, 4, null,
            boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0181817275523031, 0.0132444059746035, 0.01778000000000161, true, 1

            Part.InsertSplitLineProject(true, true);

            swSelMgr = (SelectionMgr)Part.SelectionManager;

            boolstatus = Part.Extension.SelectByID2("Split Line1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);

            myFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSplitLine = (SplitLineFeatureData)myFeature.GetDefinition();

            // Get split line feature data
            boolstatus = swSplitLine.AccessSelections(Part, null);

            Debug.Print(myFeature.Name);
            Debug.Print("    Split type as defined in swSplitLineFeatureType_e: " + swSplitLine.GetType());
            Debug.Print("    Single Direction? " + swSplitLine.SingleDirection);
        }
    }
}
```

```
        Debug.Print("    Reversed? " + swSplitLine.ReverseDirection);
        swSplitLine.ReleaseSelectionAccess();
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Create Projection Split Line Feature Example (VBA)

This example shows how to create a projection split line feature.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with a feature extrusion, reference plane,
'    and sketch of an ellipse.
' 2. Creates Split Line1 in the FeatureManager design tree.
' 3. Inspect the Immediate window.
'-----
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim skSegment As SldWorks.SketchSegment
Dim myRefPlane As SldWorks.RefPlane
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSplitLine As SldWorks.SplitLineFeatureData
Dim vSkLines As Variant
Dim myFeature As SldWorks.Feature
Dim boolstatus As Boolean
Dim longstatus As Long

Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Set Part = swApp.ActiveDoc

    Set skSegment = Part.SketchManager.CreateEllipse(-2.12512457655407E-02, 1.22505076014363E-02, 0, 2.77468345541365E-03,
7.05202391259263E-03, 0, -1.96159170237913E-02, 1.98085370103935E-02, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True

    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

    Set myRefPlane = Part.FeatureManager.InsertRefPlane(8, 0.01778, 0, 0, 0, 0)
    Part.ClearSelection2 True

    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", -4.07148636658249E-02, 2.47341229458697E-02, 1.94913387248102E-02,
False, 0, Nothing, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    boolstatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)

    vSkLines = Part.SketchManager.CreateCornerRectangle(-6.25406077424486E-02, 2.97244912047745E-02, 0, 5.84903577919818E-02,
-0.018090206988802, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False,
1.74532925199433E-02, 1.74532925199433E-02, False, False, True, True, True, 0, 0, False)
    Part.SelectionManager.EnableContourSelection = False

    boolstatus = Part.Extension.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", -1.43044793836914E-02, 3.34438727079956E-03, 0, True, 4, Nothing,
0)
    boolstatus = Part.Extension.SelectByID2("", "FACE", -1.81817275523031E-02, 1.32444059746035E-02, 1.778000000000161E-02, True, 1,
Nothing, 0)

    Part.InsertSplitLineProject True, True

    Set swSelMgr = Part.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData

    boolstatus = Part.Extension.SelectByID2("Split Line1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

    Set myFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swSplitLine = myFeature.GetDefinition

    ' Get split line feature data
    boolstatus = swSplitLine.AccessSelections(Part, Nothing)

    Debug.Print myFeature.Name
    Debug.Print "    Split type as defined in swSplitLineFeatureType_e: " & swSplitLine.GetType

```

```
Debug.Print "    Single Direction? " & swSplitLine.SingleDirection  
Debug.Print "    Reversed? " & swSplitLine.ReverseDirection
```

```
swSplitLine.ReleaseSelectionAccess
```

```
End Sub
```

Create Projection Split Line Feature Example (VB.NET)

This example shows how to create a projection split line feature.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'-----
' Postconditions:
' 1. Creates a new model document with a feature extrusion, reference plane,
'    and sketch of an ellipse.
' 2. Creates Split Line1 in the FeatureManager design tree.
' 3. Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim skSegment As SketchSegment
    Dim myRefPlane As RefPlane
    Dim swSelMgr As SelectionMgr
    Dim swSplitLine As SplitLineFeatureData
    Dim vSkLines As Object
    Dim myFeature As Feature
    Dim boolstatus As Boolean
    Dim longstatus As Integer

    Sub main()

        Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
        Part = swApp.ActiveDoc

        skSegment = Part.SketchManager.CreateEllipse(-0.0212512457655407, 0.0122505076014363, 0, 0.00277468345541365, 0.007052023912)
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)

        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

        myRefPlane = Part.FeatureManager.InsertRefPlane(8, 0.01778, 0, 0, 0, 0)
        Part.ClearSelection2(True)

        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", -0.0407148636658249, 0.0247341229458697, 0.0194913387248102, False, 0, Nothing, 0)
        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceToggle_e.swSketchAddConstToRectEntity)
        boolstatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType)

        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0625406077424486, 0.0297244912047745, 0, 0.0584903577919818, -0.01805)
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)
        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

        myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0)
        Part.SelectionManager.EnableContourSelection = False

        boolstatus = Part.Extension.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", -0.0143044793836914, 0.00334438727079956, 0, True, 4, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0181817275523031, 0.0132444059746035, 0.017780000000000161, True, 1, Nothing, 0)

        Part.InsertSplitLineProject(True, True)

        swSelMgr = Part.SelectionManager

        boolstatus = Part.Extension.SelectByID2("Split Line1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

        myFeature = swSelMgr.GetSelectedObject6(1, -1)
        swSplitLine = myFeature.GetDefinition

        ' Get split line feature data
        boolstatus = swSplitLine.AccessSelections(Part, Nothing)

        Debug.Print(myFeature.Name)
        Debug.Print("    Split type as defined in swSplitLineFeatureType_e: " & swSplitLine.GetType)
        Debug.Print("    Single Direction? " & swSplitLine.SingleDirection)
        Debug.Print("    Reversed? " & swSplitLine.ReverseDirection)

        swSplitLine.ReleaseSelectionAccess()

    End Sub

''' <summary>

```

```
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Create PropertyManager Page and Selectable Triad Manipulator Example (VBA)

This example shows how to create a PropertyManager page and a triad manipulator you can select.

```

'-----
' Preconditions:
' 1. Click Tools > References > SolidWorks <version> exposed type libraries for  

'   add-in use > OK.
' 2. Copy Macros to the main module and rename it Macros. (To rename the module,  

'   click View > Properties Window and type Macros in (Name).)
' 3. Click Insert > Class Module and copy PropMgr to that class module and  

'   name it PropMgr.
' 4. Click Insert > Class Module and copy PropMgrHdlr to that class module  

'   and name it PropMgrHdlr.
' 5. Click Insert > Class Module and copy swDragManipHdlr to that class module  

'   and name it swDragManipHdlr.
' 6. Open public_documents\samples\tutorial\api\assem20.sldasm.
' 7. Select the top face of the left-side component.
'
' Postconditions:
' 1. Displays the Sample PropertyManager page.
' 2. Click Select Entity to show the ID of the face  

'   selected in Preconditions step 7.
'   a. Click OK to close the message box.
'   b. Examine the Immediate window.
' 3. Selects the top face of the right-side component.
' 4. Click Create triad manipulator to create a triad  

'   manipulator at the pick point on the top face  

'   of the right-side component.
'   a. Right-click the triad manipulator.
'   b. Examine the Immediate window.
' 5. Click OK one or two times to close the  

'   PropertyManager page.
' 6. Click any handle on the triad manipulator.
' 7. Examine the Immediate window.
' 8. Press F5 to finish executing the macro and  

'   remove the triad manipulator.
'-----
'Macros
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc

' Application's PropertyManager page
Dim swPage As PropMgr

Sub main()

    Dim nRetVal As Long
    Set swApp = CreateObject("SldWorks.Application")

    ' Make sure that there is a model open to which to add the  

    ' PropertyManager page; if there is no model, then get rid  

    ' of references to any previous pages
    If swApp.ActiveDoc Is Nothing Then
        Set swPart = swApp.NewPart
        Set swPage = Nothing
    End If

    ' If there is no PropertyManager page, then create a new one and show it,  

    ' or if there is a PropertyManager page, then show it
    If swPage Is Nothing Then
        Set swPage = New PropMgr
        swPage.Show 'Display it
    Else
        swPage.Show
    End If
End Sub

'PropMgr
' This class defines the PropertyManager page and its controls
Option Explicit

' PropertyManager page
Private m_Page As SldWorks.PropertyManagerPage2

' Two groups contains the controls
Private m_Group As SldWorks.PropertyManagerPageGroup

' Controls on the page
Private m_Text As SldWorks.PropertyManagerPageTextbox
Private m_Check As SldWorks.PropertyManagerPageCheckbox
Private m_Selection As SldWorks.PropertyManagerPageSelectionbox
Private m_ClearSelection As SldWorks.PropertyManagerPageCheckbox
Private m_Button As SldWorks.PropertyManagerPageButton
Private m_Button1 As SldWorks.PropertyManagerPageButton
Private m_Label As SldWorks.PropertyManagerPageLabel

' To determine whether the second group is active
Private m_bGroup As Boolean

' IDs for all of the controls
Const ID_GROUP As Long = 1
Const ID_SELECTION As Long = 2
Const ID_BUTTON As Long = 3

```

[Back to top](#)

```

Const ID_BUTTON1 As Long = 4
Const ID_LABEL As Long = 5

' Generate the PropertyManager page and its controls
Dim swManip As SldWorks.Manipulator
Dim swDrag As SldWorks.DragArrowManipulator
Dim swTriad As SldWorks.TriadManipulator
Dim swDragHdlr As swDragManipHdlr
Public swFace As SldWorks.Face2

' Create the page and place all of the controls on it
Private Sub Layout()
    Dim swApp As SldWorks.SldWorks

    ' Objects needed to create the PropertyManager page
    Dim pageHdlr As New PropMgrHdlr
    Dim swPage As SldWorks.PropertyManagerPage2
    Dim swControl As SldWorks.PropertyManagerPageControl
    Dim title As String
    Dim message As String
    Dim caption As String
    Dim tip As String
    Dim listItems(3) As String
    Dim buttonTypes As Long
    Dim id As Long
    Dim options As Long
    Dim filterArray(2) As Long
    Dim errors As Long
    Dim controlType As Integer
    Dim alignment As Integer
    Dim bRet As Boolean

    ' Access SOLIDWORKS Design
    Set swApp = GetObject(, "SldWorks.Application")

    ' Initialize the Property page handler
    ' Pass a reference to this PropertyManager page
    pageHdlr.Init Me

    ' Set some variables for the PropertyManager page
    title = "Sample PropertyManager"
    buttonTypes = swPropertyManager_OkayButton + swPropertyManager_CancelButton

    ' Create the PropertyManager page
    Set m_Page = swApp.CreatePropertyManagerPage(title, buttonTypes, pageHdlr, errors)

    ' Make sure that the PropertyManger page was correctly created
    If errors = swPropertyManagerPage_Okay Then
        ' Initial setup of the dialog
        Message = "Information message that can be displayed as necessary."
        m_Page.SetMessage Message, swImportantMessageBox
        ' Begin adding the required controls to the dialog
        'GROUP Box -----
        id = ID_GROUP
        caption = "Group &1"
        options = swGroupBoxOptions_Visible + swGroupBoxOptions_Checkbox + swGroupBoxOptions_Expanded + swGroupBoxOptions_Checked
        m_bGroup = False ' Mark that the second group is disabled
        Set m_Group = m_Page.AddGroupBox(id, caption, options)
        If Not m_Group Is Nothing Then
            ' Place these controls in the second group
            'CONTROL Label -----
            id = ID_LABEL
            controlType = swControlType_Label
            caption = "Selection box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Label"
            Set swControl = m_Group.AddControl(id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Label = swControl
            End If

            'CONTROL Selection box -----
            id = ID_SELECTION
            controlType = swControlType_Selectionbox
            caption = "Sample selection box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Select faces and vertices"
            Set swControl = m_Group.AddControl(id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Selection = swControl
                filterArray(0) = swSelFACES
                filterArray(1) = swSelVERTICES
                filterArray(2) = swSelMANIPULATORS
                m_Selection.SetSelectionFilters (filterArray)
                m_Selection.AllowMultipleSelectOfSameEntity = True
                m_Selection.Height = 50
                m_Selection.Mark = 1
            End If

            'CONTROL Button -----
            id = ID_BUTTON
            controlType = swControlType_Button
            caption = "Create triad manipulator"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Button"
            Set swControl = m_Group.AddControl(id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Button = swControl
            End If
        End If
    End If
End Sub

```

```

        'CONTROL Button -----
        Id = ID_BUTTON1
        controlType = swControlType_Button
        caption = "Select Entity"
        alignment = swControlAlign_Indent
        options = swControlOptions_Visible + swControlOptions_Enabled
        tip = "Button"
        Set swControl = m_Group.AddControl(Id, controlType, caption, alignment, options, tip)
        If Not swControl Is Nothing Then
            Set m_Button1 = swControl
        End If
    End If
Else
    swApp.SendMsgToUser2 "There was an error creating the PropertyManager page.", swMbInformation, swMbOk
End If
End Sub

' Display the Property Manager page
Public Sub Show()
    Dim nRetVal As Long
    Dim bRet As Boolean

    nRetVal = m_Page.Show
    bRet = m_Page.SetMessage("Some message string.", swAnnotationVisible)
    Debug.Assert bRet
End Sub

Private Sub Class_Initialize()
    Layout
End Sub

Private Sub Class_Terminate()
End Sub

' Callback called by PropertyManagerPage2Handler_OnButtonPress
' in the PropertyManager page handler class
' This implementation just clears the selections
Public Sub OnButtonPressed(ByVal Id As Long)
    Dim swApp As SldWorks.SldWorks
    Dim swMathUtil As SldWorks.MathUtility
    Dim swModel As SldWorks.ModelDoc2
    Dim swModViewMgr As SldWorks.ModelViewManager
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim vPickPt As Variant
    Dim swPickPt As SldWorks.MathPoint
    Dim origin As SldWorks.MathPoint
    Dim pt As Variant
    Dim obj As Object
    Dim boolstatus As Boolean
    Dim nVector(2) As Double
    Dim vVector As Variant
    Dim swN As SldWorks.MathVector

    Set swDragHdlr = New swDragManipHdlr
    Set swApp = Application.SldWorks
    Set swMathUtil = swApp.GetMathUtility
    Set swModel = swApp.ActiveDoc

    If Id = ID_BUTTON1 Then
        Set swSelMgr = swModel.SelectionManager
        Id = swSelMgr.GetSelectedObjectCount2(-1)
        Set obj = swSelMgr.GetSelectedObject6(1, -1)
        If obj Is Nothing Then
            swApp.SendMsgToUser2 "Not selected.", swMbWarning, swMbOk
        Else
            swApp.SendMsgToUser2 "Selected, ID = " & swSelMgr.GetSelectedObjectType2(1), swMbWarning, swMbOk
            Debug.Print "SelType      = " & swSelMgr.GetSelectedObjectType2(1)
        End If
        Exit Sub
    End If

    swModel.ClearSelection2 True

    boolstatus = swModel.Extension.SelectByRay(0.112447854353888, 0.084630970938349, -7.93916624701296E-02, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 3.53102873173539E-03, 2, False, 0, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    nVector(0) = 0:    nVector(1) = 1:    nVector(2) = 0
    vVector = nVector
    Set swN = swMathUtil.CreateVector((vVector))
    vPickPt = swSelMgr.GetSelectionPoint(1)
    Set swPickPt = swMathUtil.CreatePoint((vPickPt))
    Set swModViewMgr = swModel.ModelViewManager
    If Id = ID_BUTTON Then
        Set swManip = swModViewMgr.CreateManipulator(swManipulatorType_e.swTriadManipulator, swDragHdlr)
        Set swTriad = swManip.GetSpecificManipulator
        swTriad.origin = swPickPt
        swManip.Show swModel
        swManip.Selectable = True
        Set origin = swTriad.origin
        pt = origin.ArrayData
        Stop
        swManip.Remove
    End If
End Sub

```

[Back to top](#)

```

'PropMgrHdlr
' This file is the implementation of the PropertyManagerPage2Handler5
' interface
' Each of these methods is called when its corresponding

```

```

' notification is fired from within SOLIDWORKS Design
' It is customary to have notifications use callback
' functions defined within the main application to perform
' any desired actions
Option Explicit

Implements PropertyManagerPage2Handler5
' Variable that provides access to the PropertyManager page
Dim m_PageObj As PropMgr

' This method is called to initialize the handler
Public Sub Init(pageObj As PropMgr)
    Set m_PageObj = pageObj
End Sub

Private Sub Class_Initialize()
    Debug.Print "Class_Initialize"
End Sub
Private Sub Class_Terminate()
    Debug.Print "Class_Terminate"
End Sub
'These methods are the implementations of the SOLIDWORKS Design notifications
Private Sub PropertyManagerPage2Handler5_AfterClose()
    Debug.Print "PropertyManagerPage2Handler5_AfterClose"
End Sub
Private Function PropertyManagerPage2Handler5_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
    Debug.Print "PropertyManagerPage2Handler5_OnActiveXControlCreated"
End Function

Private Sub PropertyManagerPage2Handler5_OnButtonPress(ByVal Id As Long)
    m_PageObj.OnButtonPressed (Id)
End Sub
Private Sub PropertyManagerPage2Handler5_OnCancel()
    Debug.Print "PropertyManagerPage2Handler5_OnCancel"
End Sub
Private Sub PropertyManagerPage2Handler5_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)
    Debug.Print "PropertyManagerPage2Handler5_OnCheckboxCheck"
End Sub
Private Sub PropertyManagerPage2Handler5_OnClose(ByVal Reason As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnClose"
End Sub
Private Sub PropertyManagerPage2Handler5_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)
    Debug.Print "PropertyManagerPage2Handler5_OnComboboxEditChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnComboboxSelectionChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)
    Debug.Print "PropertyManagerPage2Handler5_OnGroupCheck"
End Sub
Private Sub PropertyManagerPage2Handler5_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)
    Debug.Print "PropertyManagerPage2Handler5_OnGroupExpand"
End Sub
Private Function PropertyManagerPage2Handler5_OnHelp() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnHelp"
End Function
Private Sub PropertyManagerPage2Handler5_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnListboxSelectionChanged"
End Sub
Private Function PropertyManagerPage2Handler5_OnNextPage() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnNextPage"
End Function
Private Sub PropertyManagerPage2Handler5_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "PropertyManagerPage2Handler5_OnNumberboxChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnOK()
    Debug.Print "PropertyManagerPage2Handler5_OnOK"
End Sub
Private Sub PropertyManagerPage2Handler5_OnOptionCheck(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnOptionCheck"
End Sub
Private Function PropertyManagerPage2Handler5_OnPreviousPage() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnPreviousPage"
End Function
Private Sub PropertyManagerPage2Handler5_OnSelectionboxCalloutCreated(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxCalloutCreated"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSelectionboxCalloutDestroyed(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxCalloutDestroyed"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSelectionboxFocusChanged(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxFocusChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxListChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
    Debug.Print "PropertyManagerPage2Handler5_OnTextboxChanged"
End Sub
Private Function PropertyManagerPage2Handler5_OnPreview() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnPreview"
End Function
Private Function PropertyManagerPage2Handler5_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ItemText As String) As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnSubmitSelection"
End Function
Private Function PropertyManagerPage2Handler5_OnTabClicked(ByVal Id As Long) As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnTabClicked"
End Function
Private Sub PropertyManagerPage2Handler5_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "PropertyManagerPage2Handler5_OnSliderPositionChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnWhatsNew()

```

```

        Debug.Print "PropertyManagerPage2Handler5_OnWhatsNew"
    End Sub
    Private Sub PropertyManagerPage2Handler5_OnUndo()
        Debug.Print "PropertyManagerPage2Handler5_OnUndo"
    End Sub
    Private Sub PropertyManagerPage2Handler5_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
        Debug.Print "PropertyManagerPage2Handler5_OnSliderTrackingCompleted"
    End Sub
    Private Function PropertyManagerPage2Handler5_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long) As Boolean
        Debug.Print "PropertyManagerPage2Handler5_OnKeystroke"
    End Function
    Private Sub PropertyManagerPage2Handler5_OnPopupMenuUpdate(ByVal Id As Long, retVal As Long)
        Debug.Print "PropertyManagerPage2Handler5_OnPopupMenuUpdate"
    End Sub
    Private Sub PropertyManagerPage2Handler5_OnPopupMenu(ByVal Id As Long)
        Debug.Print "PropertyManagerPage2Handler5_OnPopupMenu"
    End Sub
    Private Sub PropertyManagerPage2Handler5_AfterActivation()
        Debug.Print "PropertyManagerPage2Handler5_AfterActivation"
    End Sub

```

[Back to top](#)

```

'swDragManipHdlr
Option Explicit
Implements SwManipulatorHandler2

Private Sub Class_Initialize()
    End Sub
    Private Function SwManipulatorHandler2_OnDelete(ByVal pManipulator As Object) As Boolean
        Debug.Print "SwManipulatorHandler2_OnDelete"
    End Function
    Private Sub SwManipulatorHandler2_OnDirectionFlipped(ByVal pManipulator As Object)
        Debug.Assert False
        Debug.Print "SwManipulatorHandler2_OnDirectionFlipped"
    End Sub
    Private Function SwManipulatorHandler2_OnDoubleValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As Double) As Boolean
        Debug.Assert False
        Debug.Print "SwManipulatorHandler2_OnDoubleValueChanged"
        Debug.Print "    ID          = " & Id
        Debug.Print "    Value       = " & Value
    End Function
    Private Sub SwManipulatorHandler2_OnEndDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
        Debug.Print "SwManipulatorHandler2_OnEndDrag"
        Debug.Print "    HandleIndex = " & handleIndex
        If (handleIndex = swDragArrowManipulatorOptions_e.swDragArrowManipulatorDirection1) Then
            Debug.Print "    Forward"
        Else
            Debug.Print "    Backward"
        End If
    End Sub
    Private Sub SwManipulatorHandler2_OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
        Debug.Print "SwManipulatorHandler2_OnEndNoDrag"
        Debug.Print "    HandleIndex = " & handleIndex
    End Sub
    Private Function SwManipulatorHandler2_OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean
        End Function
    Private Sub SwManipulatorHandler2_OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
        Debug.Print "SwManipulatorHandler2_OnHandleRmbSelected"
        Debug.Print "    handleIndex = " & handleIndex
    End Sub
    Private Sub SwManipulatorHandler2_OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
        Debug.Print "SwManipulatorHandler2_OnHandleSelected"
        Debug.Print "    HandleIndex = " & handleIndex
    End Sub
    Private Sub SwManipulatorHandler2_OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Long)
        Debug.Assert False
        Debug.Print "SwManipulatorHandler2_OnItemSetFocus"
        Debug.Print "    ID          = " & Id
    End Sub
    Private Function SwManipulatorHandler2_OnStringValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As String) As Boolean
        Debug.Assert False
        Debug.Print "SwManipulatorHandler2_OnStringValueChanged"
        Debug.Print "    ID          = " & Id
        Debug.Print "    Value       = " & Value
    End Function
    Private Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Long, ByVal newPosMathPt As Object)
        Debug.Print "SwManipulatorHandler2_OnUpdateDrag"
        Debug.Print "    HandleIndex = " & handleIndex
    End Sub

```

[Back to top](#)

Create PropertyManager Page Example (C#)

This example shows how to create a PropertyManager page that contains these controls:

- ActiveX
- Bitmap
- Bitmap buttons
- Button
- Combo box
- Group box
- Label
- List box
- Number box
- Radio button
- Selection box
- Slider
- Tab

This example also shows how to handle focus events for these controls.

NOTE: If the model is an assembly that contains multiple components and you want to allow the user to select edges, faces, or vertices, then you must specify `swSelectType_e.swSelCOMPSDONT OVERRIDE` for parameter `SelType` of `IPropertyManagerPageSelectionbox::SetSelectionFilters`. Otherwise, if the user attempts to select an edge, face, or vertex, then the entire component might get selected and not the edge, face, or vertex. This example demonstrates how to specify `SelType`.

```
//-----
// Preconditions:
// 1 Create a C# macro named CreatePropertyManagerPageExample_CSharp.
// 2. Copy and paste Main into SolidWorksMacro.cs.
// 3. Add a class named clsPropMgr and copy and paste clsPropMgr into that class.
// 4. Click Project > Add Reference, browse to install_dir\api\redist\, select
// SolidWorks.Interop.swpublished.dll > OK.
// 5. Verify that the specified assembly document exists.
// 6. Modify ClassID and LicenseKey parameters in
// IPropertyManagerPageActiveX::SetClass to add your ActiveX control
// to the PropertyManager page.
// 7. Replace the ImageList and ImageListMasks array elements
// with the pathnames of your image files.
// 8. Open an Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly.
// 2. Creates a PropertyManager page.
// 3. Creates the specified controls.
// 4. Examine the Immediate Window as you use the specified
// controls. Note the size of the button at the bottom
// of the PropertyManager page.
// 5. Click OK to close the PropertyManager page.
// 6. Exit SOLIDWORKS Design.
// 7. Change the size of the text and other items on your computer.
// For example, in Windows 7:
// a. Click Start > Control Panel > Appearance and
// Personalization > Display.
// b. Select a different size.
// c. Click Apply.
// d. Click Log off now.
// e. Log back in.
// 8. Start SOLIDWORKS Design.
// 9. Run this macro again and note the size of the button at
// the bottom of the PropertyManager page.
// 10. Click OK to close the PropertyManager page.
//
// NOTES:
// * After running this macro, select
// Tools > Options > System Options > Stop VSTA debugger
// on macro exit.
// * Because the assembly document is used elsewhere,
// do not save changes.
//-----

//Main
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace CreatePropertyManagerPageExample_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public ModelDoc2 Part;
        public clsPropMgr pm;

        public void Main()
        {
```

```

        int openDocErrors = 0;
        int openDocWarnings = 0;

        swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, false);

        Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\bladed
shaft.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref openDocErrors, ref
openDocWarnings);

        //Create a new instance of the PropertyManager class
        pm = new clsPropMgr(swApp);
        pm.Show();
    }

    public SldWorks swApp;
}
}

```

[Back to top](#)

```

//clsPropMgr
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace CreatePropertyManagerPageExample_CSharp.csproj
{
    [ComVisibleAttribute(true)]
    public class clsPropMgr : PropertyManagerPage2Handler9
    {
        //Control objects required for the PropertyManager page
        PropertyManagerPage2 pm_Page;
        PropertyManagerPageGroup pm_Group;
        PropertyManagerPageSelectionbox pm_Selection;
        PropertyManagerPageSelectionbox pm_Selection2;
        PropertyManagerPageLabel pm_Label;
        PropertyManagerPageCombobox pm_Combo;
        PropertyManagerPageListbox pm_List;
        PropertyManagerPageNumberbox pm_Number;
        PropertyManagerPageOption pm_Radio;
        PropertyManagerPageSlider pm_Slider;
        PropertyManagerPageTab pm_Tab;
        PropertyManagerPageButton pm_Button;
        PropertyManagerPageBitmapButton pm_BMPButton;
        PropertyManagerPageBitmapButton pm_BMPButton2;
        PropertyManagerPageBitmap pm_Bitmap;
        PropertyManagerPageActiveX pm_ActiveX;

        //Each control in the page needs a unique ID
        const int GroupID = 1;
        const int LabelID = 2;
        const int SelectionID = 3;
        const int ComboID = 4;
        const int ListID = 5;
        const int Selection2ID = 6;
        const int NumberID = 7;
        const int RadioID = 8;
        const int SliderID = 9;
        const int TabID = 10;
        const int ButtonID = 11;
        const int BMPButtonID = 12;
        const int BMPButtonID2 = 13;
        const int BitmapID = 14;
        const int ActiveXID = 15;

        public void Show()
        {
            pm_Page.Show2(0);
        }

        //The following runs when a new instance
        //of the class is created
        public clsPropMgr(SldWorks swApp)
        {
            string PageTitle = null;
            string caption = null;
            string tip = null;
            int options = 0;
            int longerrors = 0;
            int controlType = 0;

```

```

int alignment = 0;
string[] listItems = new string[4];

//Set the variables for the page
PageTitle = "Comps";
options = (int)swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton +
(int)swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton +
(int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage +
(int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton;

//Create the PropertyManager page
pm_Page = (PropertyManagerPage2)swApp.CreatePropertyManagerPage(PageTitle, (int)options, this, ref longererrors);

//Make sure that the page was created properly
if (longererrors == (int)swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay)
{
    //Add the controls to the page
    //Add a tab
    pm_Tab = pm_Page.AddTab(TabID, "Application", "", 0);

    //Add a group box to the tab
    caption = "Controls";

options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded;
    pm_Group = (PropertyManagerPageGroup)pm_Tab.AddGroupBox(GroupID, caption, options);

    //Add two selection boxes
    controlType = (int)swPropertyManagerPageControlType_e.swControlType_Selectionbox;
    caption = "";

    alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
    options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
    tip = "Select an edge, face, vertex, solid body, or a component";
    pm_Selection = (PropertyManagerPageSelectionbox)pm_Group.AddControl2(SelectionID, (short)controlType, caption,
(short)alignment, (int)options, tip);
    pm_Selection2 = (PropertyManagerPageSelectionbox)pm_Group.AddControl2(Selection2ID, (short)controlType, caption,
(short)alignment, (int)options, tip);

    swSelectType_e[] filters = new swSelectType_e[7];
    filters[0] = swSelectType_e.swSelEDGES;
    filters[1] = swSelectType_e.swSelREFEDGES;
    filters[2] = swSelectType_e.swSelFACES;
    filters[3] = swSelectType_e.swSelVERTICES;
    filters[4] = swSelectType_e.swSelSOLIDBODIES;
    filters[5] = swSelectType_e.swSelCOMPONENTS;
    filters[6] = swSelectType_e.swSelCOMPSDONTOVERRIDE;

    object filterObj = null;
    filterObj = filters;

    pm_Selection.SingleEntityOnly = false;
    pm_Selection.AllowMultipleSelectOfSameEntity = true;
    pm_Selection.Height = 50;
    pm_Selection.SetSelectionFilters(filterObj);

    pm_Selection2.SingleEntityOnly = false;
    pm_Selection2.AllowMultipleSelectOfSameEntity = true;
    pm_Selection2.Height = 50;
    pm_Selection2.SetSelectionFilters(filterObj);

    //Add a combo box
    controlType = (int)swPropertyManagerPageControlType_e.swControlType_Combobox;
    caption = "";
    alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
    options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
    tip = "Select a value";
    pm_Combo = (PropertyManagerPageCombobox)pm_Group.AddControl2(ComboID, (short)controlType, caption, (short)alignment,
(int)options, tip);
    if ((pm_Combo != null))
    {
        pm_Combo.Height = 50;
        listItems[0] = "Value 1";
        listItems[1] = "Value 2";
        listItems[2] = "Value 3";
        listItems[3] = "Value 4";
        pm_Combo.AddItem(listItems);
        pm_Combo.CurrentSelection = 0;
    }

    //Add a list box
    controlType = (int)swPropertyManagerPageControlType_e.swControlType_Listbox;
    caption = "";
    alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
    options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;

```

```

        tip = "Multi-select values in the list box";
        pm_List = (PropertyManagerPageListbox)pm_Group.AddControl2(ListID, (short)controlType, caption, (short)alignment,
(int)options, tip);

        if ((pm_List != null))
        {
            pm_List.Style = (int)swPropMgrPageListBoxStyle_e.swPropMgrPageListBoxStyle_MultipleItemSelect;
            pm_List.Height = 50;
            listItems[0] = "Value 1";
            listItems[1] = "Value 2";
            listItems[2] = "Value 3";
            listItems[3] = "Value 4";
            pm_List.AddItems(listItems);
            pm_List.SetSelectedItem(1, true);
        }

        //Add a label
        pm_Label = (PropertyManagerPageLabel)pm_Group.AddControl2(LabelID, (int)swPropertyManagerPageControlType_e.swControlType_Lat

        //Add a slider

        pm_Slider = (PropertyManagerPageSlider)pm_Group.AddControl2(SliderID, (int)swPropertyManagerPageControlType_e.swControlType_Slider,

        //Add a radio button

        pm_Radio = (PropertyManagerPageOption)pm_Group.AddControl2(RadioID, (int)swPropertyManagerPageControlType_e.swControlType_Option, "F

        //Add a number box

        pm_Number = (PropertyManagerPageNumberbox)pm_Group.AddControl2(NumberID, (int)swPropertyManagerPageControlType_e.swControlType_Numbe

        //Add a button

        pm_Button = (PropertyManagerPageButton)pm_Group.AddControl2(ButtonID, (int)swPropertyManagerPageControlType_e.swControlType_Button,

        //Add a bitmap button

        pm_BMPButton = (PropertyManagerPageBitmapButton)pm_Group.AddControl2(BMPButtonID, (int)swPropertyManagerPageControlType_e.swControl
        pm_BMPButton.SetStandardBitmaps((int)swPropertyManagerPageBitmapButtons_e.swBitmapButtonImage_parallel);

        //Add another bitmap button that scales with computer's resolution
        pm_BMPButton2 = (PropertyManagerPageBitmapButton)pm_Group.AddControl2(BMPButtonID2,
(int)swPropertyManagerPageControlType_e.swControlType_BitmapButton, "Bitmap button", (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge,
options, "Click");
        string[] imageList = new string[3];
        string[] imageListMasks = new string[3];
        imageList[0] = "Pathname_to_nxn_image";
        imageList[1] = "Pathname_to_nnxnn_image";
        imageList[2] = "Pathname_to_nnnxnnn_image";
        imageListMasks[0] = "Pathname_to_mask_nxn_image";
        imageListMasks[1] = "Pathname_to_mask_nnxnn_image";
        imageListMasks[2] = "Pathname_to_mask_nnnxnnn_image";
        pm_BMPButton2.SetBitmapsByName3(imageList, imageListMasks);

        //Add a bitmap

        pm_Bitmap = (PropertyManagerPageBitmap)pm_Group.AddControl2(BitmapID, (int)swPropertyManagerPageControlType_e.swControlType_Bitmap, "Bitmap", (int)swPrope
rtyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge, options, "Bitmap");
        pm_Bitmap.SetStandardBitmap((int)swBitmapControlStandardTypes_e.swBitmapControl_Volume);

        //Add an ActiveX control

        pm_ActiveX = (PropertyManagerPageActiveX)pm_Group.AddControl2(ActiveXID, (int)swPropertyManagerPageControlType_e.swControlType_ActiveX, "ActiveX", (int)sw
Proper tyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge, options, "ActiveX control tip");
        pm_ActiveX.SetClass("ClassID", "LicenseKey");
    }
    else
    {
        //If the page is not created
        System.Windows.Forms.MessageBox.Show("An error occurred while attempting to create the PropertyManager page.");
    }
}

}

#region IPropertyManagerPage2Handler9 Members
void IPropertyManagerPage2Handler9.AfterActivation()
{
}

void IPropertyManagerPage2Handler9.AfterClose()
{
}

int IPropertyManagerPage2Handler9.OnActiveXControlCreated(int Id, bool Status)
{
    Debug.Print("ActiveX control created");
    return 0;
}
}

```

```

void IPropertyManagerPage2Handler9.OnButtonPress(int Id)
{
    Debug.Print("Button clicked");
}
void IPropertyManagerPage2Handler9.OnCheckboxCheck(int Id, bool Checked)
{
}
void IPropertyManagerPage2Handler9.OnClose(int Reason)
{
    if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel)
    {
        Debug.Print ("Cancel button clicked");
    }
    else if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay)
    {
        Debug.Print ("OK button clicked");
    }
}
void IPropertyManagerPage2Handler9.OnComboboxEditChanged(int Id, string Text)
{
}
void IPropertyManagerPage2Handler9.OnComboboxSelectionChanged(int Id, int Item)
{
}
void IPropertyManagerPage2Handler9.OnGainedFocus(int Id)
{
    short[] varArray = null;
    Debug.Print("Control box " + Id + " gained focus");
    varArray = (short[])pm_List.GetSelectedItems();
    pm_Combo.CurrentSelection = varArray[0];
}
void IPropertyManagerPage2Handler9.OnGroupCheck(int Id, bool Checked)
{
}
void IPropertyManagerPage2Handler9.OnGroupExpand(int Id, bool Expanded)
{
}
bool IPropertyManagerPage2Handler9.OnHelp()
{
    return false;
}
bool IPropertyManagerPage2Handler9.OnKeystroke(int Wparam, int Message, int Lparam, int Id)
{
    return false;
}
void IPropertyManagerPage2Handler9.OnListboxSelectionChanged(int Id, int Item)
{
}
void IPropertyManagerPage2Handler9.OnLostFocus(int Id)
{
    Debug.Print("Control box " + Id + " lost focus");
}
bool IPropertyManagerPage2Handler9.OnNextPage()
{
    return false;
}
void IPropertyManagerPage2Handler9.OnNumberboxChanged(int Id, double Value)
{
    Debug.Print("Number box changed");
}
void IPropertyManagerPage2Handler9.OnOptionCheck(int Id)
{
    Debug.Print("Option selected");
}
void IPropertyManagerPage2Handler9.OnPopupMenuItem(int Id)

```

```

{
}
void IPropertyManagerPage2Handler9.OnPopupMenuItemUpdate(int Id, ref int retval)
{
}
bool IPropertyManagerPage2Handler9.OnPreview()
{
    return false;
}
bool IPropertyManagerPage2Handler9.OnPreviousPage()
{
    return false;
}
void IPropertyManagerPage2Handler9.OnRedo()
{
}
void IPropertyManagerPage2Handler9.OnSelectionboxCalloutCreated(int Id)
{
}
void IPropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed(int Id)
{
}
void IPropertyManagerPage2Handler9.OnSelectionboxFocusChanged(int Id)
{
    Debug.Print("The focus moved to selection box " + Id);
}
void IPropertyManagerPage2Handler9.OnSelectionboxListChanged(int Id, int Count)
{
    pm_Page.SetCursor((int)swPropertyManagerPageCursors_e.swPropertyManagerPageCursors_Advance);
    Debug.Print("The list in selection box " + Id + " changed");
}
void IPropertyManagerPage2Handler9.OnSliderPositionChanged(int Id, double Value)
{
    Debug.Print("Slider position changed");
}
void IPropertyManagerPage2Handler9.OnSliderTrackingCompleted(int Id, double Value)
{
}
bool IPropertyManagerPage2Handler9.OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
{
    // This method must return true for selections to occur
    return true;
}
bool IPropertyManagerPage2Handler9.OnTabClicked(int Id)
{
    return false;
}
void IPropertyManagerPage2Handler9.OnTextboxChanged(int Id, string Text)
{
}
void IPropertyManagerPage2Handler9.OnUndo()
{
}
void IPropertyManagerPage2Handler9.OnWhatsNew()
{
}
void IPropertyManagerPage2Handler9.OnListboxRMBUp(int Id, int PosX, int PosY)
{
}
int IPropertyManagerPage2Handler9.OnWindowFromHandleControlCreated(int Id, bool Status)
{
    return 0;
}
}

```

```
void IPropertyManagerPage2Handler9.OnNumberBoxTrackingCompleted(int Id, double Value)
{
}
#endregion
}
```

[Back to top](#)

Create PropertyManager Page Example (VBA)

This example shows how to create a PropertyManager page that contains these controls:

- ActiveX
- Bitmap
- Bitmap buttons
- Button
- Combo box
- Group box
- Label
- List box
- Number box
- Radio button
- Selection box
- Slider
- Tab

This example also shows how to handle focus events for these controls.

NOTE: If the model is an assembly that contains multiple components, and you want to allow the user to select edges, faces, or vertices, then you must specify `swSelectType_e.swSelCOMPSDONT OVERRIDE` for parameter `SelType` of `IPropertyManagerPageSelectionbox::SetSelectionFilters`. Otherwise, if the user attempts to select an edge, face, or vertex, then the entire component might get selected and not the edge, face, or vertex. This example demonstrates how to specify `SelType`.

```
'-----  
' Preconditions:  
' 1. Copy and paste Main into your project.  
' 2. Insert a class and copy and paste Class1 into that class.  
' 3. Click Tools > References, select  
'    SOLIDWORKS version exposed type libraries for add-in use, and click OK.  
' 4. Verify that the specified assembly document exists.  
' 5. Open an Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified assembly.  
' 2. Creates a PropertyManager page.  
' 3. Creates the specified controls.  
' 4. Examine the Immediate Window as you use the specified controls.  
'    Note the size of the button at the bottom of the PropertyManager page.  
' 5. Click OK to close the PropertyManager page.  
' 6. Exit SOLIDWORKS Design.  
'  
' NOTE: Because the assembly document is used elsewhere,  
' do not save changes  
'-----  
  
'Main  
Option Explicit  
  
Public swApp As SldWorks.SldWorks  
Public Part As SldWorks.ModelDoc2  
Public pm As Class1  
  
Sub main()  
  
    Dim openDocErrors As Long  
    Dim OpenDocWarnings As Long  
  
    Set swApp = Application.SldWorks  
    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm", swDocASSEMBLY,  
swOpenDocOptions_Silent, "", openDocErrors, OpenDocWarnings)  
  
    'Create a new instance of the PropertyManager class  
    Set pm = New Class1  
    pm.Show  
  
End Sub
```

[Back to top](#)

```
'Class1  
  
Option Explicit  
  
' Handler for PropertyManager page controls  
Implements PropertyManagerPage2Handler9  
  
' Control objects required for the PropertyManager page  
Dim pm_Page As PropertyManagerPage2  
Dim pm_Group As PropertyManagerPageGroup  
Dim pm_Selection As PropertyManagerPageSelectionbox  
Dim pm_Selection2 As PropertyManagerPageSelectionbox  
Dim pm_Label As PropertyManagerPageLabel  
Dim pm_Combo As PropertyManagerPageCombobox  
Dim pm_List As PropertyManagerPageListbox  
Dim pm_Number As PropertyManagerPageNumberbox  
Dim pm_Radio As PropertyManagerPageOption  
Dim pm_Slider As PropertyManagerPageSlider  
Dim pm_Tab As PropertyManagerPageTab  
Dim pm_Button As PropertyManagerPageButton
```

```

Dim pm_BMPButton As PropertyManagerPageBitmapButton
Dim pm_BMPButton2 As PropertyManagerPageBitmapButton
Dim pm_Bitmap As PropertyManagerPageBitmap
Dim pm_ActiveX As PropertyManagerPageActiveX
Dim ClickedCancel As Boolean
Dim retVal As Long

' Each control in the page needs a unique ID
Const GroupID As Long = 1
Const LabelID As Long = 2
Const SelectionID As Long = 3
Const ComboID As Long = 4
Const ListID As Long = 5
Const Selection2ID As Long = 6
Const NumberID As Long = 7
Const RadioID As Long = 8
Const SliderID As Long = 9
Const TabID As Long = 10
Const ButtonID As Long = 11
Const BMPButtonID As Long = 12
Const BMPButtonID2 As Long = 13
Const BitmapID As Long = 14
Const ActiveXID As Long = 15

Sub Show()

    pm_Page.Show2 0

End Sub

' Run when a new instance of the class is created
Private Sub Class_Initialize()

    Dim PageTitle As String
    Dim caption As String
    Dim tip As String
    Dim options As Long
    Dim longerrors As Long
    Dim controlType As Long
    Dim alignment As Long
    Dim listItems(3) As String

    ' Set the variables for the page
    PageTitle = "Comps"

    options = swPropertyManager_OkayButton _
        + swPropertyManager_CancelButton _
        + swPropertyManagerOptions_LockedPage _
        + swPropertyManagerOptions_PushpinButton

    ' Create the PropertyManager page
    Set pm_Page = swApp.CreatePropertyManagerPage(PageTitle, _
        options, Me, longerrors)

    ' Make sure that the page was created properly
    If longerrors = swPropertyManagerPage_Okay Then

        ' Add controls to the page

        ' Add a tab
        Set pm_Tab = pm_Page.AddTab(TabID, "Application", "", 0)

        ' Add a group box to the tab
        caption = "Controls"
        options = swGroupBoxOptions_Visible + _
            swGroupBoxOptions_Expanded
        Set pm_Group = pm_Tab.AddGroupBox(GroupID, caption, options)

        ' Add two selection boxes
        controlType = swControlType_Selectionbox
        caption = ""
        alignment = swControlAlign_Indent
        options = swControlOptions_Visible + _
            swControlOptions_Enabled
        tip = "Select an edge, face, vertex, solid body, or a component"
        Set pm_Selection = pm_Group.AddControl2(SelectionID, _
            controlType, caption, alignment, options, tip)

        Set pm_Selection2 = pm_Group.AddControl2(Selection2ID, _
            controlType, caption, alignment, options, tip)

        Dim filters(6) As Long
        filters(0) = swSelEDGES
        filters(1) = swSelREFEDGES
        filters(2) = swSelFACES
        filters(3) = swSelVERTICES
        filters(4) = swSelSOLIDBODIES
        filters(5) = swSelCOMPONENTS
        filters(6) = swSelCOMPSONTOVERRIDE

        pm_Selection.SingleEntityOnly = False
        pm_Selection.AllowMultipleSelectOfSameEntity = True
        pm_Selection.Height = 50
        pm_Selection.SetSelectionFilters filters
        pm_Selection2.SingleEntityOnly = False
        pm_Selection2.AllowMultipleSelectOfSameEntity = True
    
```

```

pm_Selection2.Height = 50
pm_Selection2.SetSelectionFilters filters

' Add a combo box
controlType = swControlType_Combobox
caption = ""
alignment = swControlAlign_Indent
options = swControlOptions_Visible + _
           swControlOptions_Enabled
tip = "Select a value"

Set pm_Combo = pm_Group.AddControl2(ComboID, _
    controlType, caption, alignment, options, tip)

If Not pm_Combo Is Nothing Then
    pm_Combo.Height = 50

    listItems(0) = "Value 1"
    listItems(1) = "Value 2"
    listItems(2) = "Value 3"
    listItems(3) = "Value 4"

    pm_Combo.AddItem (listItems)
    pm_Combo.CurrentSelection = 0

End If

' Add a list box
controlType = swControlType_Listbox
caption = ""
alignment = swControlAlign_Indent
options = swControlOptions_Visible + _
           swControlOptions_Enabled
tip = "Multi-select values in the list box"

Set pm_List = pm_Group.AddControl2(ListID, _
    controlType, caption, alignment, options, tip)
pm_List.Style = swPropMgrPageListBoxStyle_MultipleItemSelect
pm_List.Height = 50

If Not pm_List Is Nothing Then
    pm_List.Height = 50
    listItems(0) = "Value 1"
    listItems(1) = "Value 2"
    listItems(2) = "Value 3"
    listItems(3) = "Value 4"
    pm_List.AddItem (listItems)
    pm_List.SetSelectedItem 1, True

End If

' Add a label
Set pm_Label = pm_Group.AddControl2(LabelID, swControlType_Label, "Label", swControlAlign_LeftEdge, options, "")

' Add a slider
Set pm_Slider = pm_Group.AddControl2(SliderID, swControlType_Slider, "Slider", swControlAlign_LeftEdge, options, "Slide")

' Add a radio button
Set pm_Radio = pm_Group.AddControl2(RadioID, swControlType_Option, "Radio button", swControlAlign_LeftEdge, options,
"Select")

' Add a number box
Set pm_Number = pm_Group.AddControl2(NumberID, swControlType_Numberbox, "Number box", swControlAlign_LeftEdge, options,
"Spin")

' Add a button
Set pm_Button = pm_Group.AddControl2(ButtonID, swControlType_Button, "Button", swControlAlign_LeftEdge, options, "Click")

' Add a bitmap button
Set pm_BMPButton = pm_Group.AddControl2(BMPButtonID, swControlType_BitmapButton, "Bitmap button", swControlAlign_LeftEdge,
options, "Click")
pm_BMPButton.SetStandardBitmaps (swPropertyManagerPageBitmapButtons_e.swBitmapButtonImage_parallel)

' Add a bitmap
Set pm_Bitmap = pm_Group.AddControl2(BitmapID, swControlType_Bitmap, "Bitmap", swControlAlign_LeftEdge, options, "Bitmap")
pm_Bitmap.SetStandardBitmap (swBitmapControlStandardTypes_e.swBitmapControl_Volume)

' To add an ActiveX control, uncomment the following two lines and modify ClassID and LicenseKey parameters for your ActiveX
control.
' Set pm_ActiveX = pm_Group.AddControl2(ActiveXID, swControlType_ActiveX, "ActiveX", swControlAlign_LeftEdge, options,
"ActiveX control tip")
' pm_ActiveX.SetClass "ClassID", "LicenseKey"

```

```

        ' To add another bitmap button that scales with your computer's resolution, uncomment the 10 code lines below and replace
the
        ' ImageList and ImageListMasks array elements with the pathnames of scaled image files. After running this macro, change the
        ' screen resolution of your computer. Log off and back in. Start SOLIDWORKS Design and run this macro again. Notice how this
button
        ' control scales with screen resolution.

        ' Set pm_BMPButton2 = pm_Group.AddControl2(BMPButtonID2, swControlType_BitmapButton, "Bitmap button",
swControlAlign_LeftEdge, options, "Click")
        ' Dim imageList(2) As String
        ' Dim imageListMasks(2) As String
        ' imageList(0) = "Pathname_to_nxn_image"
        ' imageList(1) = "Pathname_to_nnxnn_image"
        ' imageList(2) = "Pathname_to_nnnxnnn_image"
        ' imageListMasks(0) = "Pathname_to_mask_nxn_image"
        ' imageListMasks(1) = "Pathname_to_mask_nnxnn_image"
        ' imageListMasks(2) = "Pathname_to_mask_nnnxnnn_image"
        ' pm_BMPButton2.SetBitmapsByName3 imageList, imageListMasks
    Else
        MsgBox "An error occurred while attempting to create the PropertyManager Page", vbCritical
    End If

End Sub

Private Sub PropertyManagerPage2Handler9_AfterActivation()
End Sub

Private Sub PropertyManagerPage2Handler9_AfterClose()
    ' Destroy the class
    Set pm = Nothing
End Sub

Private Function PropertyManagerPage2Handler9_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
    Debug.Print "ActiveX control created"
End Function

Private Sub PropertyManagerPage2Handler9_OnButtonPress(ByVal Id As Long)
    Debug.Print "Button clicked"
End Sub

Private Sub PropertyManagerPage2Handler9_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler9_OnClose(ByVal Reason As Long)
    If Reason = swPropertyManagerPageClose_Cancel Then
        ' Cancel button clicked
        ClickedCancel = True
    ElseIf Reason = swPropertyManagerPageClose_Okay Then
        ' OK button clicked
        ClickedCancel = False
    End If
End Sub

Private Sub PropertyManagerPage2Handler9_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)
End Sub

Private Sub PropertyManagerPage2Handler9_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler9_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)
End Sub

Private Function PropertyManagerPage2Handler9_OnHelp() As Boolean
End Function

Private Function PropertyManagerPage2Handler9_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal
Id As Long) As Boolean
End Function

```

```

Private Sub PropertyManagerPage2Handler9_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)

End Sub

Private Function PropertyManagerPage2Handler9_OnNextPage() As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "Number box changed"
End Sub

Private Sub PropertyManagerPage2Handler9_OnOptionCheck(ByVal Id As Long)
    Debug.Print "Option selected"
End Sub

Private Sub PropertyManagerPage2Handler9_OnPopupMenuItem(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnPopupMenuItemUpdate(ByVal Id As Long, retVal As Long)
End Sub

Private Function PropertyManagerPage2Handler9_OnPreview() As Boolean
End Function

Private Function PropertyManagerPage2Handler9_OnPreviousPage() As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnRedo()
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxCalloutCreated(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxCalloutDestroyed(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxFocusChanged(ByVal Id As Long)
    Debug.Print "The focus moved to selection box " & Id
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)
    pm_Page.SetCursor (swPropertyManagerPageCursors_Advance)
    Debug.Print "The list in selection box " & Id & " changed"
End Sub

Private Sub PropertyManagerPage2Handler9_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "Slider position changed"
End Sub

Private Sub PropertyManagerPage2Handler9_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub

Private Function PropertyManagerPage2Handler9_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long,
ItemText As String) As Boolean
    PropertyManagerPage2Handler9_OnSubmitSelection = True
End Function

Private Function PropertyManagerPage2Handler9_OnTabClicked(ByVal Id As Long) As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
End Sub

Private Sub PropertyManagerPage2Handler9_OnUndo()
End Sub

Private Sub PropertyManagerPage2Handler9_OnWhatsNew()

```

End Sub

Private Sub PropertyManagerPage2Handler9_OnLostFocus(ByVal Id As Long)

 Debug.Print "Control box " & Id & " lost focus"

End Sub

Private Sub PropertyManagerPage2Handler9_OnGainedFocus(ByVal Id As Long)

 Dim varArray As Variant

 Debug.Print "Control box " & Id & " gained focus"

 varArray = pm_List.**GetSelectedItems**

 pm_Combo.**CurrentSelection** = varArray(0)

End Sub

Public Sub PropertyManagerPage2Handler9_OnListBoxRMBUp(ByVal Id As Long, ByVal posX As Long, ByVal posY As Long)

End Sub

Public Function PropertyManagerPage2Handler9_OnWindowFromHandleControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long

End Function

Public Sub PropertyManagerPage2Handler9_OnNumberboxTrackingCompleted(ByVal Id As Long, ByVal Value As Double)

End Sub

[Back to top](#)

Create PropertyManager Page Example (VB.NET)

This example shows how to create a PropertyManager page that contains these controls:

- ActiveX
- Bitmap
- Bitmap buttons
- Button
- Combo box
- Group box
- Label
- List box
- Number box
- Radio button
- Selection box
- Slider
- Tab

This example also shows how to handle focus events for these controls.

NOTE: If the model is an assembly that contains multiple components, and you want to allow the user to select edges, faces, or vertices, then you must specify `swSelectType_e.swSelCOMPSDONTOVERRIDE` for parameter `SelType` of `IPropertyManagerPageSelectionbox::SetSelectionFilters`. Otherwise, if the user attempts to select an edge, face, or vertex, then the entire component might get selected and not the edge, face, or vertex. This example demonstrates how to specify `SelType`.

```

'-----
' Preconditions:
' 1. Create a VB.NET macro.
' 2. Copy and paste Main into SolidWorksMacro.vb.
' 3. Add a class named clsPropMgr and copy and paste clsPropMgr into that class.
' 4. Click Project > Add Reference, browse to install_dir\api\redist\, select
' SolidWorks.Interop.swpublished.dll > OK.
' 5. Verify that the specified assembly document exists.
' 6. Modify ClassID and LicenseKey parameters in
' IPropertyManagerPageActiveX::SetClass to add your ActiveX control
' to the PropertyManager page.
' 7. Replace the ImageList and ImageListMasks array elements with the pathnames of
' your image files.
' 8. Open an Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Creates a PropertyManager page.
' 3. Creates the specified controls.
' 4. Examine the Immediate Window as you use the specified controls.
' Note the size of the button at the bottom of the PropertyManager page.
' 5. Click OK to close the PropertyManager page.
' 6. Exit SOLIDWORKS Design.
' 7. Change the size of the text and other items on your computer.
' For example, in Windows 7:
' a. Click Start > Control Panel > Appearance and
' Personalization > Display.
' b. Select a different size.
' c. Click Apply.
' d. Click Log off now.
' e. Log back in.
' 8. Start SOLIDWORKS Design.
' 9. Run this macro again and note the size of the button at
' the bottom of the PropertyManager page.
' 10. Click OK to close the PropertyManager page.
'
' NOTES:
' * After running this macro, select
' Tools > Options > System Options > Stop VSTA debugger
' on macro exit.
' * Because the assembly document is used elsewhere,
' do not save any changes.
'-----

'Main
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro
    Public Part As ModelDoc2
    Public WithEvents pm As clsPropMgr

    Sub main()
        Dim openDocErrors As Integer
        Dim OpenDocWarnings As Integer

        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, False)

        Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm",
        swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", openDocErrors, OpenDocWarnings)
    End Sub
End Class

```

```

        'Create a new instance of the PropertyManager class
        pm = New clsPropMgr(swApp)
        pm.Show()
    End Sub

    Public swApp As SldWorks
End Class

```

[Back to top](#)

```

'clsPropMgr
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Runtime.InteropServices
Imports System.Diagnostics

<ComVisibleAttribute(True)> _
Public Class clsPropMgr

    Implements PropertyManagerPage2Handler9

    Dim swApp As SldWorks

    'Control objects required for the PropertyManager page
    Dim pm_Page As PropertyManagerPage2
    Dim pm_Group As PropertyManagerPageGroup
    Dim pm_Selection As PropertyManagerPageSelectionbox
    Dim pm_Selection2 As PropertyManagerPageSelectionbox
    Dim pm_Label As PropertyManagerPageLabel
    Dim pm_Combo As PropertyManagerPageCombobox
    Dim pm_List As PropertyManagerPageListbox
    Dim pm_Number As PropertyManagerPageNumberbox
    Dim pm_Radio As PropertyManagerPageOption
    Dim pm_Slider As PropertyManagerPageSlider
    Dim pm_Tab As PropertyManagerPageTab
    Dim pm_Button As PropertyManagerPageButton
    Dim pm_BMPButton As PropertyManagerPageBitmapButton
    Dim pm_BMPButton2 As PropertyManagerPageBitmapButton
    Dim pm_Bitmap As PropertyManagerPageBitmap
    Dim pm_ActiveX As PropertyManagerPageActiveX

    'Each control in the page needs a unique ID
    Const GroupID As Integer = 1
    Const LabelID As Integer = 2
    Const SelectionID As Integer = 3
    Const ComboID As Integer = 4
    Const ListID As Integer = 5
    Const Selection2ID As Integer = 6
    Const NumberID As Integer = 7
    Const RadioID As Integer = 8
    Const SliderID As Integer = 9
    Const TabID As Integer = 10
    Const ButtonID As Integer = 11
    Const BMPButtonID As Integer = 12
    Const BMPButtonID2 As Integer = 13
    Const BitmapID As Integer = 14
    Const ActiveXID As Integer = 15

    Dim ClickedCancel As Boolean
    Dim retVal As Integer

    Sub Show()
        pm_Page.Show2(0)
    End Sub

    'Run when a new instance
    'of the class is created
    Public Sub New(ByVal swApp As SldWorks)

        Dim PageTitle As String
        Dim caption As String
        Dim tip As String
        Dim options As Integer
        Dim longerrors As Integer
        Dim controlType As Integer
        Dim alignment As Integer
        Dim listItems(3) As String

        'Set the variables for the page
        PageTitle = "Comps"

        options = swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton _
            + swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton _
            + swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage _
            + swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton

        'Create the PropertyManager page
        pm_Page = CType(swApp.CreatePropertyManagerPage(PageTitle, _
            options, Me, longerrors), PropertyManagerPage2)
    End Sub
End Class

```

```
'Make sure that the page was created properly
If longerrors = swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay Then
```

```
'Add the controls to the page
'Add a tab
pm_Tab = pm_Page.AddTab(TabID, "Application", "", 0)
```

```
'Add a group box to the tab
caption = "Controls"
options = swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + _
    swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded
pm_Group = pm_Tab.AddGroupBox(GroupID, caption, options)
```

```
'Add two selection boxes
controlType = swPropertyManagerPageControlType_e.swControlType_Selectionbox
caption = "" ' No caption for selection boxes
alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
options = swAddControlOptions_e.swControlOptions_Visible + _
    swAddControlOptions_e.swControlOptions_Enabled
tip = "Select an edge, face, vertex, solid body, or a component"
pm_Selection = pm_Group.AddControl2(SelectionID, _
    controlType, caption, alignment, options, tip)
pm_Selection2 = pm_Group.AddControl2(Selection2ID, _
    controlType, caption, alignment, options, tip)
Dim filters(6) As swSelectType_e
filters(0) = swSelectType_e.swSelEDGES
filters(1) = swSelectType_e.swSelREFEDGES
filters(2) = swSelectType_e.swSelFACES
filters(3) = swSelectType_e.swSelVERTICES
filters(4) = swSelectType_e.swSelSOLIDBODIES
filters(5) = swSelectType_e.swSelCOMPONENTS
filters(6) = swSelectType_e.swSelCOMPSDONTTOVERRIDE
Dim filterObj As Object
filterObj = filters
pm_Selection.SingleEntityOnly = False
pm_Selection.AllowMultipleSelectOfSameEntity = True
pm_Selection.Height = 50
pm_Selection.SetSelectionFilters(filterObj)
pm_Selection2.SingleEntityOnly = False
pm_Selection2.AllowMultipleSelectOfSameEntity = True
pm_Selection2.Height = 50
pm_Selection2.SetSelectionFilters(filterObj)
```

```
'Add a combo box
controlType = swPropertyManagerPageControlType_e.swControlType_Combobox
caption = ""
alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
options = swAddControlOptions_e.swControlOptions_Visible + _
    swAddControlOptions_e.swControlOptions_Enabled
```

```
tip = "Select a value"
pm_Combo = pm_Group.AddControl2(ComboID, _
    controlType, caption, alignment, options, tip)
If Not pm_Combo Is Nothing Then
    pm_Combo.Height = 50
    listItems(0) = "Value 1"
    listItems(1) = "Value 2"
    listItems(2) = "Value 3"
    listItems(3) = "Value 4"
    pm_Combo.AddItem(listItems)
    pm_Combo.CurrentSelection = 0
End If
```

```
'Add a list box
controlType = swPropertyManagerPageControlType_e.swControlType_Listbox
caption = ""
alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
options = swAddControlOptions_e.swControlOptions_Visible + _
    swAddControlOptions_e.swControlOptions_Enabled
tip = "Multi-select values in the list box"
pm_List = pm_Group.AddControl2(ListID, _
    controlType, caption, alignment, options, tip)
pm_List.Style = swPropMgrPageListBoxStyle_e.swPropMgrPageListBoxStyle_MultipleItemSelect
pm_List.Height = 50
If Not pm_List Is Nothing Then
    pm_List.Height = 50
    listItems(0) = "Value 1"
    listItems(1) = "Value 2"
    listItems(2) = "Value 3"
    listItems(3) = "Value 4"
    pm_List.AddItem(listItems)
    pm_List.SetSelectedItem(1, True)
End If
```

```
'Add a label
pm_Label = pm_Group.AddControl2(LabelID, swPropertyManagerPageControlType_e.swControlType_Label, "Label", swPropertyManz
```

```
'Add a slider
pm_Slider = pm_Group.AddControl2(SliderID, swPropertyManagerPageControlType_e.swControlType_Slider, "Slider", swProperty
```

```

'Add a radio button
pm_Radio = pm_Group.AddControl2(RadioID, swPropertyManagerPageControlType_e.swControlType_Option, "Radio button", swProp

'Add a number box
pm_Number = pm_Group.AddControl2(NumberID, swPropertyManagerPageControlType_e.swControlType_Numberbox, "Number box", swF

'Add a button
pm_Button = pm_Group.AddControl2(ButtonID, swPropertyManagerPageControlType_e.swControlType_Button, "Button", swProperty

'Add a bitmap button
pm_BMPButton = pm_Group.AddControl2(BMPButtonID, swPropertyManagerPageControlType_e.swControlType_BitmapButton, "Bitmap
pm_BMPButton.SetStandardBitmaps(swPropertyManagerPageBitmapButtons_e.swBitmapButtonImage_parallel)

' Add another bitmap button that scales with your computer's resolution
pm_BMPButton2 = pm_Group.AddControl2(BMPButtonID2, swPropertyManagerPageControlType_e.swControlType_BitmapButton, "Bitmap button",
swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge, options, "Click")
Dim imageList(2) As String
Dim imageListMasks(2) As String
imageList(0) = "Pathname_to_nxn_image"
imageList(1) = "Pathname_to_nnnnn_image"
imageList(2) = "Pathname_to_nnnnnnn_image"
imageListMasks(0) = "Pathname_to_mask_nxn_image"
imageListMasks(1) = "Pathname_to_mask_nnnnn_image"
imageListMasks(2) = "Pathname_to_mask_nnnnnnn_image"
pm_BMPButton2.SetBitmapsByName3(imageList, imageListMasks)

'Add a bitmap
pm_Bitmap = pm_Group.AddControl2(BitmapID, swPropertyManagerPageControlType_e.swControlType_Bitmap, "Bitmap", swPropertyManagerPageControlLeft
Align_e.swControlAlign_LeftEdge, options, "Bitmap")
pm_Bitmap.SetStandardBitmap(swBitmapControlStandardTypes_e.swBitmapControl_Volume)

'Add an ActiveX control
pm_ActiveX = pm_Group.AddControl2(ActiveXID, swPropertyManagerPageControlType_e.swControlType_ActiveX, "ActiveX", swPropertyManagerPageControl
LeftAlign_e.swControlAlign_LeftEdge, options, "ActiveX control tip")
pm_ActiveX.SetClass("ClassID", "LicenseKey")

Else
    MsgBox("An error occurred while attempting to create the PropertyManager Page", vbCritical)
End If
End Sub

Public Sub AfterActivation() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterActivation
End Sub

Public Sub AfterClose() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterClose
End Sub

Public Function OnActiveXControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnActiveXControlCreated
    Debug.Print("ActiveX control created")
End Function

Public Sub OnButtonPress(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnButtonPress
    Debug.Print("Button clicked")
End Sub

Public Sub OnCheckboxCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnCheckboxCheck
End Sub

Public Sub OnClose(ByVal Reason As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnClose
    If Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel Then
        'Cancel button clicked
        ClickedCancel = True
    ElseIf Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay Then
        'OK button clicked
        ClickedCancel = False
    End If
End Sub

Public Sub OnComboboxEditChanged(ByVal Id As Integer, ByVal Text As String) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnComboboxEditChanged
End Sub

Public Sub OnComboboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnComboboxSelectionChanged

```

End Sub

Public Sub OnGainedFocus(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnGainedFocus**

Dim varArray As Object

Debug.Print("Control box " & Id & " gained focus")

varArray = pm_List.**GetSelectedItems**

pm_Combo.**CurrentSelection** = varArray(0)

End Sub

Public Sub OnGroupCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnGroupCheck**

End Sub

Public Sub OnGroupExpand(ByVal Id As Integer, ByVal Expanded As Boolean) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnGroupExpand**

End Sub

Public Function OnHelp() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnHelp**

End Function

Public Function OnKeystroke(ByVal Wparam As Integer, ByVal Message As Integer, ByVal Lparam As Integer, ByVal Id As Integer) As
Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnKeystroke**

End Function

Public Sub OnListboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnListboxSelectionChanged**

End Sub

Public Sub OnLostFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnLostFocus**

Debug.Print("Control box " & Id & " lost focus")

End Sub

Public Function OnNextPage() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnNextPage**

End Function

Public Sub OnNumberboxChanged(ByVal Id As Integer, ByVal Value As Double) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnNumberboxChanged**

Debug.Print("Number box changed")

End Sub

Public Sub OnOptionCheck(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnOptionCheck**

Debug.Print ("Option selected")

End Sub

Public Sub OnPopupMenuItem(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnPopupMenuItem**

End Sub

Public Sub OnPopupMenuItemUpdate(ByVal Id As Integer, ByRef retval As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnPopupMenuItemUpdate**

End Sub

Public Function OnPreview() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnPreview**

End Function

Public Function OnPreviousPage() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnPreviousPage**

End Function

Public Sub OnRedo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnRedo**

End Sub

Public Sub OnSelectionboxCalloutCreated(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.**OnSelectionboxCalloutCreated**

End Sub

```

    Public Sub OnSelectionboxCalloutDestroyed(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed
    End Sub

    Public Sub OnSelectionboxFocusChanged(ByVal Id As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxFocusChanged
        Debug.Print("The focus moved to selection box " & Id)
    End Sub

    Public Sub OnSelectionboxListChanged(ByVal Id As Integer, ByVal Count As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxListChanged
        pm_Page.SetCursor(swPropertyManagerPageCursors_e.swPropertyManagerPageCursors_Advance)
        Debug.Print("The list in selection box " & Id & " changed")
    End Sub

    Public Sub OnSliderPositionChanged(ByVal Id As Integer, ByVal Value As Double) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSliderPositionChanged
        Debug.Print("Slider position changed")
    End Sub

    Public Sub OnSliderTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSliderTrackingCompleted
    End Sub

    Public Function OnSubmitSelection(ByVal Id As Integer, ByVal Selection As Object, ByVal SelType As Integer, ByRef ItemText As
String) As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSubmitSelection
        Return True
    End Function

    Public Function OnTabClicked(ByVal Id As Integer) As Boolean Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnTabClicked
    End Function

    Public Sub OnTextboxChanged(ByVal Id As Integer, ByVal Text As String) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnTextboxChanged
    End Sub

    Public Sub OnUndo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnUndo
    End Sub

    Public Sub OnWhatsNew() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWhatsNew
    End Sub

    Public Sub OnListBoxRMBUp(ByVal Id As Integer, ByVal posX As Integer, ByVal posY As Integer) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnListBoxRMBUp
    End Sub

    Public Function OnWindowFromHandleControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWindowFromHandleControlCreated
    End Function

    Public Sub OnNumberBoxTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements
SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNumberBoxTrackingCompleted
    End Sub
End Class

```

[Back to top](#)

Create PropertyManager Page With Many Controls Example (VBA)

This example shows how to create a PropertyManager page with many controls.

```
'-----
' Preconditions:
' 1. Copy Main into your macro.
' 2. Click Insert > Class Module, name it PropMgr, and copy PropMgr
' into the module. (To rename the module, click View >
' Properties Window and type PropMgr in (Name).
' 3. Click Insert > Class Module, name it PropMgrHdlr, and copy
' PropMgrHdlr in the module.
' 4. Add a reference to SolidWorks <version> exposed
' type libraries for add-in use.
' 5. Open a part or assembly.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Creates a PropertyManager page.
' 2. Click or type text in the different controls and
' examine the Immediate window after action.
' 3. Click OK to close the PropertyManager
' page and examine the Immediate window.
'-----
'Main
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc
```

```
'The application's PropertyManager page
Dim swPage As PropMgr
```

```
Sub main()
```

```
    Dim nRetVal As Long
    Set swApp = CreateObject("SldWorks.Application")
    'Make sure that there is a model to which to
    'the page; if there is no model, then get rid
    'of references to any previous pages
    If swApp.ActiveDoc Is Nothing Then
        Set swPart = swApp.NewPart
        Set swPage = Nothing
    End If

    'If there is no page, create a new one and show it
    'If there is a page, then show it
    If swPage Is Nothing Then
        Set swPage = New PropMgr
        swPage.Show
    Else
        swPage.Show
    End If
End Sub
```

[Back to top](#)

```
'PropMgr
'This class defines the PropertyManager page and its controls
```

Option Explicit

```
'The PropertyManager page
Private m_Page As SldWorks.PropertyManagerPage2
```

```
'The two groups that will contain the controls
Private m_Group1 As SldWorks.PropertyManagerPageGroup
Private m_Group2 As SldWorks.PropertyManagerPageGroup
```

```
'The controls on the page
Private m_Text As SldWorks.PropertyManagerPageTextbox
Private m_Check As SldWorks.PropertyManagerPageCheckbox
Private m_Option1 As SldWorks.PropertyManagerPageOption
Private m_Option2 As SldWorks.PropertyManagerPageOption
Private m_Option3 As SldWorks.PropertyManagerPageOption
Private m_List As SldWorks.PropertyManagerPageListbox
Private m_Selection1 As SldWorks.PropertyManagerPageSelectionbox
Private m_Selection2 As SldWorks.PropertyManagerPageSelectionbox
Private m_Number As SldWorks.PropertyManagerPageNumberbox
Private m_Combo As SldWorks.PropertyManagerPageCombobox
Private m_ClearSelection As SldWorks.PropertyManagerPageCheckbox
Private m_Button As SldWorks.PropertyManagerPageButton
Private m_Label As SldWorks.PropertyManagerPageLabel
Private m_pActiveXControl As SldWorks.PropertyManagerPageActiveX
```

```
'Whether the second group is active
Private m_bGroup2 As Boolean
```

```
'The IDs for all of the controls
Const ID_GROUP1 As Long = 1
Const ID_TEXT As Long = 2
Const ID_CHECK As Long = 3
Const ID_OPTION1 As Long = 4
Const ID_OPTION2 As Long = 5
Const ID_OPTION3 As Long = 6
Const ID_LIST As Long = 7
```

```

Const ID_GROUP2 As Long = 8
Const ID_SELECTION1 As Long = 9
Const ID_SELECTION2 As Long = 10
Const ID_NUMBER As Long = 11
Const ID_COMBO As Long = 12
Const ID_BUTTON As Long = 13
Const ID_LABEL As Long = 14
Const ID_ACTIVEX As Long = 15

'Create the page and place all of the controls on it
Private Sub Layout()
    Dim swApp As SldWorks.SldWorks

    'Objects needed to create the PropertyManager page
    Dim pageHdlr As New PropMgrHdlr
    Dim swPage As SldWorks.PropertyManagerPage2
    Dim swControl As SldWorks.PropertyManagerPageControl
    Dim title As String
    Dim Message As String
    Dim caption As String
    Dim tip As String
    Dim listItems(3) As String
    Dim buttonTypes As Long
    Dim Id As Long
    Dim options As Long
    Dim filterArray(1) As Long
    Dim errors As Long
    Dim controlType As Integer
    Dim alignment As Integer
    Dim bRet As Boolean

    'Access SOLIDWORKS Design
    Set swApp = GetObject(, "SldWorks.Application")

    'Initialize the page handler
    'Pass a reference to this PropertyManager page
    pageHdlr.Init Me

    'Set some variables for the page
    title = "Sample PropertyManager"
    buttonTypes = swPropertyManager_OkayButton + swPropertyManager_CancelButton

'Create the PropertyManager page
    Set m_Page = swApp.CreatePropertyManagerPage(title, buttonTypes, pageHdlr, errors)

    'Make sure that it was created properly
    If errors = swPropertyManagerPage_Okay Then

        'Initial set up of the dialog
        Message = "Information message that can be displayed as necessary."
        m_Page.SetMessage Message, swImportantMessageBox

        'Begin adding the controls to the PropertyManager page
        'GROUP BOX -----
        Id = ID_GROUP1
        caption = "Group &1"
        options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded
        Set m_Group1 = m_Page.AddGroupBox(Id, caption, options)
        If Not m_Group1 Is Nothing Then
            'Place these controls in the first group
            'CONTROL Textbox -----
            Id = ID_TEXT
            controlType = swControlType_Textbox
            caption = "Sample text box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Text box"
            Set swControl = Nothing
            Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Text = swControl
                'm_Text.Style = swconst.swPropMgrPageTextBoxStyle_e.
            End If
            'CONTROL Checkbox -----
            Id = ID_CHECK
            controlType = swControlType_Checkbox
            caption = "Sample check box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Check box"
            Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Check = swControl
                m_Check.Checked = False
            End If
            'CONTROL Option -----
            Id = ID_OPTION1
            controlType = swControlType_Option
            caption = "Radio button 1"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Option 1"
            Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Option1 = swControl
                m_Option1.Checked = True
            End If
            'CONTROL Option -----
            Id = ID_OPTION2
            controlType = swControlType_Option
            caption = "Radio button 2"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Option 2"

```

```

Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
If Not swControl Is Nothing Then
    Set m_Option2 = swControl
    m_Option2.Checked = False
End If
'CONTROL Option -----
Id = ID_OPTION3
controlType = swControlType_Option
caption = "Radio button 3"
alignment = swControlAlign_Indent
options = swControlOptions_Visible + swControlOptions_Enabled
tip = "Option 3"
Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
If Not swControl Is Nothing Then
    Set m_Option3 = swControl
    m_Option3.Checked = False
End If
'CONTROL List box -----
Id = ID_LIST
controlType = swControlType_Listbox
caption = "Sample list box"
alignment = swControlAlign_Indent
options = swControlOptions_Visible + swControlOptions_Enabled
tip = "List box"
Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
If Not swControl Is Nothing Then
    Set m_List = swControl
    m_List.Height = 50
    listItems(0) = "One Fish"
    listItems(1) = "Two Fish"
    listItems(2) = "Red Fish"
    listItems(3) = "Blue Fish"
    m_List.AddItem (listItems)
    m_List.CurrentSelection = 2
End If
'CONTROL Button -----
Id = ID_BUTTON
controlType = swControlType_Button
caption = "Sample button"
alignment = swControlAlign_Indent
options = swControlOptions_Visible + swControlOptions_Enabled
tip = "Button"
Set swControl = m_Group1.AddControl(Id, controlType, caption, alignment, options, tip)
If Not swControl Is Nothing Then
    Set m_Button = swControl
End If
End If
'GROUP BOX -----
Id = ID_GROUP2
caption = "Group &2"
options = swGroupBoxOptions_Visible + swGroupBoxOptions_Checkbox + swGroupBoxOptions_Expanded + swGroupBoxOptions_Checked
m_bGroup2 = False 'Mark that the second group is disabled
Set m_Group2 = m_Page.AddGroupBox(Id, caption, options)
If Not m_Group2 Is Nothing Then
    'Place these controls in the second group
    'CONTROL Label -----
    Id = ID_LABEL
    controlType = swControlType_Label
    caption = "Sample label"
    alignment = swControlAlign_Indent
    options = swControlOptions_Visible + swControlOptions_Enabled
    tip = "Label"
    Set swControl = m_Group2.AddControl(Id, controlType, caption, alignment, options, tip)
    If Not swControl Is Nothing Then
        Set m_Label = swControl
    End If
    'CONTROL Selection box -----
    Id = ID_SELECTION1
    controlType = swControlType_Selectionbox
    caption = "Sample selection box"
    alignment = swControlAlign_Indent
    options = swControlOptions_Visible + swControlOptions_Enabled
    tip = "Select faces and vertices"
    Set swControl = m_Group2.AddControl(Id, controlType, caption, alignment, options, tip)
    If Not swControl Is Nothing Then
        Set m_Selection1 = swControl
        filterArray(0) = swSelFACES
        filterArray(1) = swSelVERTICES
        m_Selection1.SetSelectionFilters (filterArray)
        m_Selection1.SingleEntityOnly = True
        m_Selection1.Height = 50
        m_Selection1.Mark = 1
    End If
    'CONTROL Selection box -----
    Id = ID_SELECTION2
    controlType = swControlType_Selectionbox
    caption = "Sample selection box"
    alignment = swControlAlign_Indent
    options = swControlOptions_Visible + swControlOptions_Enabled
    tip = "Select faces and vertices"
    Set swControl = m_Group2.AddControl(Id, controlType, caption, alignment, options, tip)
    If Not swControl Is Nothing Then
        Set m_Selection2 = swControl
        filterArray(0) = swSelFACES
        filterArray(1) = swSelVERTICES
        m_Selection2.SetSelectionFilters (filterArray)
        m_Selection2.SingleEntityOnly = True
        m_Selection2.Height = 50
        m_Selection2.Mark = 2
    End If
    'CONTROL Number box -----
    Id = ID_NUMBER

```

```

        controlType = swControlType_Numberbox
        caption = "Sample number box"
        alignment = swControlAlign_Indent
        options = swControlOptions_Visible + swControlOptions_Enabled
        tip = "Number box"
        Set swControl = m_Group2.AddControl(Id, controlType, caption, alignment, options, tip)
        If Not swControl Is Nothing Then
            Set m_Number = swControl
            m_Number.SetRange2 swNumberBox_Length, -0.01, 0.01, True, 0.00001, 0.0001, 0.00005
            m_Number.Value = 0.01
            m_Number.Style = swPropMgrPageNumberBoxStyle_e.swPropMgrPageNumberBoxStyle_Thumbwheel
            Debug.Print m_Number.Style
        End If
        'CONTROL Combo box -----
        Id = ID_COMBO
        controlType = swControlType_Combobox
        caption = "Sample combo list box"
        alignment = swControlAlign_Indent
        options = swControlOptions_Visible + swControlOptions_Enabled
        tip = "Combo box"
        Set swControl = m_Group2.AddControl(Id, controlType, caption, alignment, options, tip)
        If Not swControl Is Nothing Then
            Set m_Combo = swControl
            m_Combo.Height = 50
            listItems(0) = "One Fish"
            listItems(1) = "Two Fish"
            listItems(2) = "Red Fish"
            listItems(3) = "Blue Fish"
            m_Combo.AddItem (listItems)
            m_Combo.CurrentSelection = 1
        End If
    End If
Else
    swApp.SendMsgToUser2 "There was an error creating the Property Manager page.", swMbInformation, swMbOk
End If
End Sub
'Display the PropertyManager page
Public Sub Show()
    Dim nRetVal As Long

    nRetVal = m_Page.Show

End Sub
'Generate the PropertyManagre page and its controls
Private Sub Class_Initialize()
    Layout
End Sub

'Callback called by PropertyManagerPage2Handler5_OnButtonPress
'in the PropertyManagerPage2Handler5 class
'This implementation just clears the selections
Public Sub OnButtonPressed(ByVal Id As Long)
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim nSelCount As Long
    Dim i As Long

    If Id = ID_BUTTON Then
        Set swApp = GetObject(, "SldWorks.Application")
        Set swModel = swApp.ActiveDoc
        Set swSelMgr = swModel.SelectionManager
        nSelCount = swSelMgr.GetSelectedObjectCount
        For i = 1 To nSelCount
            Debug.Print "SelMark(" + Str(i) + ") =" + Str(swSelMgr.GetSelectedObjectMark(i))
        Next i
        swModel.ClearSelection2 True
    End If
End Sub

```

[Back to top](#)

```

'PropMgrHdlr
' This file is the implementation of the PropertyManagerPage2Handler5
' interface
' Each of these methods is called when its corresponding
' notification is fired from within SOLIDWORKS Design
' It is customary to have notifications use callback
' functions defined within the main application to perform
' any desired actions
Option Explicit
Implements PropertyManagerPage2Handler5
' Variable that provides access to the PropertyManager page
Dim m_PageObj As PropMgr
' This method is called to initialize the handler
Public Sub Init(pageObj As PropMgr)
    Set m_PageObj = pageObj
End Sub
Private Sub Class_Initialize()
    Debug.Print "Class_Initialize"
End Sub
Private Sub Class_Terminate()
    Debug.Print "Class_Terminate"
End Sub
'These methods are the implementations of the SOLIDWORKS Design notifications
Private Sub PropertyManagerPage2Handler5_AfterClose()
    Debug.Print "PropertyManagerPage2Handler5_AfterClose"
End Sub
Private Function PropertyManagerPage2Handler5_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
    Debug.Print "PropertyManagerPage2Handler5_OnActiveXControlCreated"
End Function
Private Sub PropertyManagerPage2Handler5_OnButtonPress(ByVal Id As Long)
    m_PageObj.OnButtonPressed (Id)

```

```

End Sub
Private Sub PropertyManagerPage2Handler5_OnCancel()
    Debug.Print "PropertyManagerPage2Handler5_OnCancel"
End Sub
Private Sub PropertyManagerPage2Handler5_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)
    Debug.Print "PropertyManagerPage2Handler5_OnCheckboxCheck"
End Sub
Private Sub PropertyManagerPage2Handler5_OnClose(ByVal Reason As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnClose"
End Sub
Private Sub PropertyManagerPage2Handler5_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)
    Debug.Print "PropertyManagerPage2Handler5_OnComboboxEditChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnComboboxSelectionChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)
    Debug.Print "PropertyManagerPage2Handler5_OnGroupCheck"
End Sub
Private Sub PropertyManagerPage2Handler5_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)
    Debug.Print "PropertyManagerPage2Handler5_OnGroupExpand"
End Sub
Private Function PropertyManagerPage2Handler5_OnHelp() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnHelp"
End Function
Private Sub PropertyManagerPage2Handler5_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnListboxSelectionChanged"
End Sub
Private Function PropertyManagerPage2Handler5_OnNextPage() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnNextPage"
End Function
Private Sub PropertyManagerPage2Handler5_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "PropertyManagerPage2Handler5_OnNumberboxChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnOK()
    Debug.Print "PropertyManagerPage2Handler5_OnOK"
End Sub
Private Sub PropertyManagerPage2Handler5_OnOptionCheck(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnOptionCheck"
End Sub
Private Function PropertyManagerPage2Handler5_OnPreviousPage() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnPreviousPage"
End Function
Private Sub PropertyManagerPage2Handler5_OnSelectionboxCalloutCreated(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxCalloutCreated"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSelectionboxCalloutDestroyed(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxCalloutDestroyed"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSelectionboxFocusChanged(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxFocusChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnSelectionboxListChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
    Debug.Print "PropertyManagerPage2Handler5_OnTextboxChanged"
End Sub
Private Function PropertyManagerPage2Handler5_OnPreview() As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnPreview"
End Function
Private Function PropertyManagerPage2Handler5_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ItemText As String) As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnSubmitSelection"
End Function
Private Function PropertyManagerPage2Handler5_OnTabClicked(ByVal Id As Long) As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnTabClicked"
End Function
Private Sub PropertyManagerPage2Handler5_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "PropertyManagerPage2Handler5_OnSliderPositionChanged"
End Sub
Private Sub PropertyManagerPage2Handler5_OnWhatsNew()
    Debug.Print "PropertyManagerPage2Handler5_OnWhatsNew"
End Sub
Private Sub PropertyManagerPage2Handler5_OnUndo()
    Debug.Print "PropertyManagerPage2Handler5_OnUndo"
End Sub
Private Sub PropertyManagerPage2Handler5_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
    Debug.Print "PropertyManagerPage2Handler5_OnSliderTrackingCompleted"
End Sub
Private Function PropertyManagerPage2Handler5_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long) As Boolean
    Debug.Print "PropertyManagerPage2Handler5_OnKeystroke"
End Function
Private Sub PropertyManagerPage2Handler5_OnPopupMenuItemUpdate(ByVal Id As Long, retVal As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnPopupMenuItemUpdate"
End Sub
Private Sub PropertyManagerPage2Handler5_OnPopupMenuItem(ByVal Id As Long)
    Debug.Print "PropertyManagerPage2Handler5_OnPopupMenuItem"
End Sub
Private Sub PropertyManagerPage2Handler5_AfterActivation()
    Debug.Print "PropertyManagerPage2Handler5_AfterActivation"
End Sub

```

[Back to top](#)

Create Radiate Surface Feature Example (C#)

This example shows how to create a radiate surface feature.

```
//-----
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new model document with a feature extrusion.
// 2. Creates Boss-Extrude1 and Surface-Radiate1 in the graphics area and
//    FeatureManager design tree.
// 3. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateSurfRadiateFeat_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr swSelMgr;
        SelectData swSelData;
        SurfaceRadiateFeatureData swRadiate;
        Feature swFeat;
        Entity swEnt;
        object[] vRadEnt;
        Entity swRadDirEnt;
        int i;

        bool boolStatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2015\\templates\\Part.prtdot", 0, 0, 0);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolStatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.0448901407839529, 0.0279954694016864, 0.0046682067411
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            boolStatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swl
            boolStatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0555749908365768, 0.0329075527136081, 0, 0.0478203409524033, -0.6
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            Part.ShowNamedView2("**Trimetric", 8);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            boolStatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);

            object myFeature = null;
            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);

            boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0447337592343047, 0.0328467250718631, 0.00258132540182032, false,
            boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0556265649287866, 0.0156695101210289, 0.0025672149453726, true, 2
            boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0140113588298618, -0.0317157034173761, 0.00254079743683633, true, 2
            boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047780958393389, -0.00542256709667299, 0.00256078163948814, true, 2
            boolStatus = Part.Extension.SelectByID2("", "FACE", 0.0478203409524554, -0.00305747564971171, 0.000546558985774936, true

            Part.InsertRadiateSurface(0.0254, false, false);

            swSelMgr = (SelectionMgr)Part.SelectionManager;
            swSelData = swSelMgr.CreateSelectData();

            boolStatus = Part.Extension.SelectByID2("Surface-Radiate1", "REFSURFACE", 0, 0, 0, false, 0, null, 0);

            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swRadiate = (SurfaceRadiateFeatureData)swFeat.GetDefinition();

            // Get radiate surface data
            Debug.Print("File = " + Part.GetPathName());
            Debug.Print(" " + swFeat.Name);
            Debug.Print(" Distance: " + swRadiate.Distance * 1000.0 + " mm");
            Debug.Print(" Flip? " + swRadiate.Flip);
            Debug.Print(" Propagate to tangent faces? " + swRadiate.PropagateToTangentFaces);
        }
    }
}
```

```

// Roll back to get direction reference and radiated edges
boolStatus = swRadiate.AccessSelections(Part, null);
swRadDirEnt = (Entity)swRadiate.DirectionReference;
Part.ClearSelection2(true);

vRadEnt = (object[])swRadiate.RadiatedEntities;

Debug.Print("Type as defined in swSelectType_e:");
for (i = 0; i <= swRadiate.GetRadiatedEntitiesCount() - 1; i++)
{
    swEnt = (Entity)vRadEnt[i];
    Debug.Print("    Radiated Entity(" + i + ") = " + swEnt.GetType());
}

swRadiate.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;

}
}

```

Create Radiate Surface Feature Example (VB.NET)

This example shows how to create a radiate surface feature.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with a feature extrusion.
' 2. Creates Boss-Extrude1 and Surface-Radiate1 in the graphics area and
'    FeatureManager design tree.
' 3. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim Part As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swSelData As SelectData
    Dim swRadiate As SurfaceRadiateFeatureData
    Dim swFeat As Feature
    Dim swEnt As Edge
    Dim vRadEnt As Object
    Dim swRadDirEnt As Entity
    Dim i As Integer
    Dim boolStatus As Boolean

    Sub main()

        Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
        Part = swApp.ActiveDoc

        boolStatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.0448901407839529, 0.0279954694016864, 0.00466820674117181
        Part.SketchManager.InsertSketch(True)
        Part.ClearSelection2(True)
        boolStatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreference
        boolStatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPrefere
        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0555749908365768, 0.0329075527136081, 0, 0.0478203409524033, -0.03171
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)
        Part.ShowNamedView2("Trimetric", 8)
        Part.SketchManager.InsertSketch(True)
        Part.ClearSelection2(True)
        boolStatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

        Dim myFeature As Object
        myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.

        boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0447337592343047, 0.0328467250718631, 0.00258132540182032, False, 2,
        boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0556265649287866, 0.0156695101210289, 0.0025672149453726, True, 2, No
        boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0140113588298618, -0.0317157034173761, 0.00254079743683633, True, 2,
        boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047780958393389, -0.00542256709667299, 0.00256078163948814, True, 2, No
        boolStatus = Part.Extension.SelectByID2("", "FACE", 0.0478203409524554, -0.00305747564971171, 0.000546558985774936, True, 1,

        Part.InsertRadiateSurface(0.0254, False, False)

        swSelMgr = Part.SelectionManager
        swSelData = swSelMgr.CreateSelectData

        boolStatus = Part.Extension.SelectByID2("Surface-Radiate1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)

        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swRadiate = swFeat.GetDefinition

        ' Get radiate surface data
        Debug.Print("File = " & Part.GetPathName)
        Debug.Print(" " & swFeat.Name)
        Debug.Print(" Distance: " & swRadiate.Distance * 1000.0# & " mm")
        Debug.Print(" Flip? " & swRadiate.Flip)
        Debug.Print(" Propagate to tangent faces? " & swRadiate.PropagateToTangentFaces)

        ' Roll back to get direction reference and radiated edges
        boolStatus = swRadiate.AccessSelections(Part, Nothing)
        swRadDirEnt = swRadiate.DirectionReference
        Part.ClearSelection2(True)

        vRadEnt = swRadiate.RadiatedEntities

        Debug.Print("Type as defined in swSelectType_e:")
    End Sub

```

```
For i = 0 To swRadiate.GetRadiatedEntitiesCount - 1
    swEnt = vRadEnt(i)
    Debug.Print("    Radiated Entity(" & i & ") = " & swEnt.GetType)
Next i

swRadiate.ReleaseSelectionAccess()
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Create Reference Plane Structure System Member Example (C#)

This example shows how to create a structure system member along the intersection of two or more planes.

```
// =====
// Preconditions:
// 1. Ensure the specified part template exists.
// 2. Open an Immediate window.
// 3. Press F5.
//
// Postconditions:
// 1. Creates five reference planes:
//    - Plane1 parallel to Right Plane
//    - Plane2 and Plane3 parallel to and flanking Top Plane
//    - Plane4 and Plane5 parallel to and flanking Front Plane
// 2. Configures the member profile.
// 3. Selects the start/end reference planes that define the lengths of the members: Plane1 and Right Plane.
// 4. Selects the axis reference planes that define the direction of the members: Plane2 and Plane3.
// 5. Selects the location reference planes that define the positions of the members: Plane4 and Plane5.
// 6. Creates a Structure System1 with nine structure system reference plane members.
// 7. Inspect the structure system in the graphics area.
// 8. Press F5 to edit the structure system.
// 9. Gets Member1.
// 10. Changes its extensions in directions 1 and 2.
// 11. Modifies the structure system.
// 12. Inspect the changes in the graphics area.
// =====
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
namespace RefPlane_CSharp
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private ModelDocExtension modDocExt;
        private StructureSystemMemberFeatureData structMemDef;
        private StructureSystemMemberProfile profDef;
        private PrimaryStructuralMemberFeatureData PrimDef;
        private PrimaryMemberRefPlaneFeatureData memRefPlaneDef;
        private object[] startEndPlanes = new object[2];
        private object[] horzPlanes = new object[3];
        private object[] vertPlanes = new object[3];
        private bool stat;
        private StructureSystemMemberFeatureData[] memData = new StructureSystemMemberFeatureData[1];
        private object[] memDataArray;
        private Feature structSysFeat;
        private Feature feat;
        private StructureSystemFolder structSysModDef;
        private object[] outProfiles;
        private StructureSystemMemberFeatureData MemberData;
        private object[] memberArray;
        private Feature profileGrpFeat;
```

```

private ProfileGroupFolder profileGrp;
private StructureSystemMemberFeatureData memToChange;
private int I;
private int j;

public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize;
    ArraySize = Objects.GetUpperBound(0);
    var d = new Dispatcher[ArraySize + 1];
    int ArrayIndex;
    var loopTo = ArraySize;
    for (ArrayIndex = 0; ArrayIndex <= loopTo; ArrayIndex++)
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    return d;
}

public void Main()
{
    double swSheetWidth;
    swSheetWidth = 0d;
    double swSheetHeight;
    swSheetHeight = 0d;
    modDoc = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0,
swSheetWidth, swSheetHeight);
    modDoc = (ModelDoc2)swApp.ActiveDoc;
    swSelMgr =(SelectionMgr) modDoc.SelectionManager;
    modDocExt = modDoc.Extension;
    stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    object myRefPlane;
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.381d, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.1778d, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(264, 0.1778d, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Plane2", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.1778d, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Plane3", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(264, 0.1778d, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Plane4", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
    stat = modDocExt.SelectByID2("Plane1", "PLANE", 0.0729275826275853d, 0.103693179609365d, 0.188906126602433d, true, 0,
null, 0);
    stat = modDocExt.SelectByID2("Plane2", "PLANE", -0.035035924945646d, 0.0882769827595098d, 0.154770056203461d, true, 0,
null, 0);
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    stat = modDocExt.SelectByID2("Plane3", "PLANE", -0.0359335022268397d, 0.0486897858681345d, 0.154489603864079d, true, 0,
null, 0);
    stat = modDocExt.SelectByID2("Plane4", "PLANE", -0.0353613843872154d, 0.122848840931965d, 0.120060929442332d, true, 0,
null, 0);
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);

```

```
    null, 0);
    stat = modDocExt.SelectByID2("Plane5", "PLANE", -0.0313617150023902d, 0.123208535607716d, 0.0798165581882131d, true, 0,
```

```
    // Create structure system reference plane member
    structMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(1);
    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
    structMemDef.StructureSystemMemberType);
    Debug.Print("Extend length in direction 1 (m): " + structMemDef.StartEndExtendD1);
    Debug.Print("Extend length in direction 2 (m): " + structMemDef.StartEndExtendD2);
    profDef = (StructureSystemMemberProfile)structMemDef.MemberProfile;
    profDef.ProfileStandard = "ansi inch";
    profDef.ProfileType = "c channel";
    profDef.ProfileSize = "3 x 5";
    PrimDef = (PrimaryStructuralMemberFeatureData)structMemDef;
    Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " +
    PrimDef.PrimaryMemberType);
    memRefPlaneDef = (PrimaryMemberRefPlaneFeatureData)PrimDef;

    // Define the member lengths
    startEndPlanes[0] = swSelMgr.GetSelectedObject6(1, 0);
    startEndPlanes[1] = swSelMgr.GetSelectedObject6(2, 0);
    DispatchWrapper[] dArray;
    dArray = ObjectArrayToDispatchWrapperArray(startEndPlanes);
    stat = memRefPlaneDef.SetStartAndEndReferences(dArray);
    Debug.Print("Reference planes set successfully: " + stat);
    horzPlanes[0] = swSelMgr.GetSelectedObject6(3, 0);
    horzPlanes[1] = swSelMgr.GetSelectedObject6(4, 0);
    horzPlanes[2] = swSelMgr.GetSelectedObject6(5, 0);
    DispatchWrapper[] eArray;
    eArray = ObjectArrayToDispatchWrapperArray(horzPlanes);
    stat = memRefPlaneDef.SetReferenceAxes(eArray);
    Debug.Print("Reference axis planes set successfully: " + stat);
    vertPlanes[0] = swSelMgr.GetSelectedObject6(6, 0);
    vertPlanes[1] = swSelMgr.GetSelectedObject6(7, 0);
    vertPlanes[2] = swSelMgr.GetSelectedObject6(8, 0);
    DispatchWrapper[] fArray;
    fArray = ObjectArrayToDispatchWrapperArray(vertPlanes);
    stat = memRefPlaneDef.SetReferenceLocations(fArray);
    Debug.Print("Reference location planes set successfully: " + stat);
    memData[0] = structMemDef;
    memDataArray = memData;
    structSysFeat = modDocExt.CreateStructureSystem(memDataArray, null);
    modDoc.ShowNamedView2("Trimetric", 8);
    modDoc.ViewZoomtofit2();
    Debugger.Break();
    structSysModDef = (StructureSystemFolder)structSysFeat.GetSpecificFeature2();
    Debug.Print("Number of profile group folders: " + structSysModDef.GetProfileGroupFoldersCount());
    outProfiles = (object[])structSysModDef.GetProfileGroupFolders();
    if (outProfiles.Length > 0)
    {
        var loopTo = outProfiles.GetUpperBound(0);
        for (I = 0; I <= loopTo; I++)
        {
            profileGrpFeat = (Feature)outProfiles[I];
            profileGrp = (ProfileGroupFolder)profileGrpFeat.GetSpecificFeature2();
            if (profileGrp is object)
            {
                Debug.Print("Number of members in profile group folder: " + profileGrp.GetStructureSystemMemberCount());
                memberArray = (object[])profileGrp.GetStructureSystemMembers();
                if (memberArray.Length > 0)
                {
                    var loopTo1 = memberArray.GetUpperBound(0);
                    for (j = 0; j <= loopTo1; j++)
```

```

        {
            MemberData = (StructureSystemMemberFeatureData)memberArray[j]; // Get first member only
            break;
        }
    }
}

memToChange = MemberData;
feat = memToChange.GetFeature();

// Change the extensions in directions 1 and 2 for Member1
memToChange.StartEndExtendD1 = 0.5d;
memToChange.StartEndExtendD2 = 0.2d;
modDoc.ClearSelection2(true);
Debug.Print("Member extensions set successfully: " + stat);

// Save edits
stat = feat.ModifyDefinition(MemberData, modDoc, null);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create Reference Plane Structure System Member Example (VB.NET)

This example shows how to create a structure system member along the intersection of two or more planes.

```
'=====
'Preconditions:
'1. Ensure the specified part template exists.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
' 1. Creates five reference planes:
'    - Plane1 parallel to Right Plane
'    - Plane2 and Plane3 parallel to and flanking Top Plane
'    - Plane4 and Plane5 parallel to and flanking Front Plane
' 2. Configures the member profile.
' 3. Selects the start/end reference planes that define the lengths of the members: Plane1 and Right Plane.
' 4. Selects the axis reference planes that define the direction of the members: Plane2 and Plane3.
' 5. Selects the location reference planes that define the positions of the members: Plane4 and Plane5.
' 6. Creates a Structure System1 with nine structure system reference plane members.
' 7. Inspect the structure system in the graphics area.
' 8. Press F5 to edit the structure system.
' 9. Gets Member1.
'10. Changes its extensions in directions 1 and 2.
'11. Modifies the structure system.
'12. Inspect the changes in the graphics area.
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim modDoc As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim modDocExt As ModelDocExtension
    Dim structMemDef As StructureSystemMemberFeatureData
    Dim profDef As StructureSystemMemberProfile
    Dim PrimDef As PrimaryStructuralMemberFeatureData
    Dim memRefPlaneDef As PrimaryMemberRefPlaneFeatureData
    Dim startEndPlanes(1) As Object
    Dim horzPlanes(2) As Object
    Dim vertPlanes(2) As Object
    Dim stat As Boolean
    Dim memData(0) As StructureSystemMemberFeatureData
    Dim memDataArray() As Object
    Dim structSysFeat As Feature
    Dim feat As Feature
    Dim structSysModDef As StructureSystemFolder
    Dim outProfiles() As Object
    Dim MemberData As StructureSystemMemberFeatureData
    Dim memberArray() As Object
    Dim profileGrpFeat As Feature
    Dim profileGrp As ProfileGroupFolder
    Dim memToChange As StructureSystemMemberFeatureData
    Dim i As Integer
    Dim j As Integer
```

```

Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As DispatchWrapper
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

Sub main()

    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    modDoc = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0, swSheetWidth,
swSheetHeight)

    modDoc = swApp.ActiveDoc

    swSelMgr = modDoc.SelectionManager
    modDocExt = modDoc.Extension

    stat = modDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

    Dim myRefPlane As Object
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.381, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.1778, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(264, 0.1778, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Plane2", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.1778, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Plane3", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(264, 0.1778, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID("Plane4", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.ClearSelection2(True)

    stat = modDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Plane1", "PLANE", 0.0729275826275853, 0.103693179609365, 0.188906126602433, True, 0, Nothing,
0)
    stat = modDocExt.SelectByID("Plane2", "PLANE", -0.035035924945646, 0.0882769827595098, 0.154770056203461, True, 0,
Nothing, 0)
    stat = modDocExt.SelectByID("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Plane3", "PLANE", -0.0359335022268397, 0.0486897858681345, 0.154489603864079, True, 0,
Nothing, 0)
    stat = modDocExt.SelectByID("Plane4", "PLANE", -0.0353613843872154, 0.122848840931965, 0.120060929442332, True, 0,
Nothing, 0)
    stat = modDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    stat = modDocExt.SelectByID("Plane5", "PLANE", -0.0313617150023902, 0.123208535607716, 0.0798165581882131, True, 0,
Nothing, 0)

```

```

'Create structure system reference plane member
structMemDef = modDocExt.CreateStructureSystemMemberData(1)
Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

Debug.Print("Extend length in direction 1 (m): " & structMemDef.StartEndExtendD1)
Debug.Print("Extend length in direction 2 (m): " & structMemDef.StartEndExtendD2)

profDef = structMemDef.MemberProfile

profDef.ProfileStandard = "ansi inch"
profDef.ProfileType = "c channel"
profDef.ProfileSize = "3 x 5"

PrimDef = structMemDef
Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
PrimDef.PrimaryMemberType)

memRefPlaneDef = PrimDef

'Define the member lengths
startEndPlanes(0) = swSelMgr.GetSelectedObject6(1, 0)
startEndPlanes(1) = swSelMgr.GetSelectedObject6(2, 0)

Dim dArray() As DispatcherWrapper
dArray = ObjectArrayToDispatcherWrapperArray(startEndPlanes)

stat = memRefPlaneDef.SetStartAndEndReferences(dArray)
Debug.Print("Reference planes set successfully: " & stat)

horzPlanes(0) = swSelMgr.GetSelectedObject6(3, 0)
horzPlanes(1) = swSelMgr.GetSelectedObject6(4, 0)
horzPlanes(2) = swSelMgr.GetSelectedObject6(5, 0)

Dim eArray() As DispatcherWrapper
eArray = ObjectArrayToDispatcherWrapperArray(horzPlanes)

stat = memRefPlaneDef.SetReferenceAxes(eArray)
Debug.Print("Reference axis planes set successfully: " & stat)

vertPlanes(0) = swSelMgr.GetSelectedObject6(6, 0)
vertPlanes(1) = swSelMgr.GetSelectedObject6(7, 0)
vertPlanes(2) = swSelMgr.GetSelectedObject6(8, 0)

Dim fArray() As DispatcherWrapper
fArray = ObjectArrayToDispatcherWrapperArray(vertPlanes)

stat = memRefPlaneDef.SetReferenceLocations(fArray)
Debug.Print("Reference location planes set successfully: " & stat)

memData(0) = structMemDef

memDataArray = memData

structSysFeat = modDocExt.CreateStructureSystem(memDataArray, Nothing)

modDoc.ShowNamedView2("**Trimetric", 8)
modDoc.ViewZoomtofit2()

Stop

```

```

structSysModDef = structSysFeat.GetSpecificFeature2
Debug.Print("Number of profile group folders: " & structSysModDef.GetProfileGroupFoldersCount)

outProfiles = structSysModDef.GetProfileGroupFolders()

If (outProfiles.Length > 0) Then
    For I = 0 To UBound(outProfiles)
        profileGrpFeat = outProfiles(I)
        profileGrp = profileGrpFeat.GetSpecificFeature2
        If Not profileGrp Is Nothing Then
            Debug.Print("Number of members in profile group folder: " & profileGrp.GetStructureSystemMemberCount)
            memberArray = profileGrp.GetStructureSystemMembers()
            If (memberArray.Length > 0) Then
                For j = 0 To UBound(memberArray)
                    MemberData = memberArray(j) 'Get first member only
                Exit For
            Next
        End If
    End If
    Next
End If

memToChange = MemberData

feat = memToChange.GetFeature()

'Change the extensions in directions 1 and 2 for Member1
memToChange.StartEndExtendD1 = 0.5
memToChange.StartEndExtendD2 = 0.2

modDoc.ClearSelection2(True)

Debug.Print("Member extensions set successfully: " & stat)

'Save edits
stat = feat.ModifyDefinition(MemberData, modDoc, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```